Ibb University
Faculty of Engineering and Architecture
Department of Electrical Engineering

University Services App for a Students in the Faculty of Engineering and Architecture, IBB University.

A project submitted to the Department of Electrical Engineering, Faculty of Engineering and Architecture, IBB UNIVERSITY, in partial fulfilment of the requirements for the award of the degree of **B.Sc. in Electronics and Computer engineering.**

By:

Abdullah Abdul Salam
Shehab Al – Saidi
Hamid Anter
Ahmed Dahan
Mohammed Bazrah

Under Supervision of:

Dr. Khalid Al-Soufi

2025

Dedication

To my first refuge in life, to those who are the reason for my existence on this earth, to the ones who love me most, to those who taught me how to be patient and how to sacrifice for others. No matter how much I try to express my gratitude, I will never be able to repay you. I pray to Allah to bless you with long lives and endless happiness, for you are the light that illuminates my path and the source of strength I draw from with every step I take.

To my happiness in this world, to those who were my support in times of hardship, to the hearts that embrace me in both my joys and sorrows. You are the ones who shared the moments of my life, a priceless blessing, and no words can express how grateful I am to have you in my life.

To the one who lit the candle of this success, to the one who planted the seeds of creativity within us and instilled in our hearts the values of knowledge and hard work. To the one who was our first support, who did not hesitate to give his wise guidance and precious time,

Dr. Khalid Al-Soufi, dear, with all my appreciation and gratitude. Because of you, we have become capable of overcoming challenges and facing the future with determination.

To the one who never closed his door to his students, to the one who diligently worked to assist anyone who turned to him, to the one who was a father, brother, and friend all at once. You are our role model, and your mark in our lives will continue to illuminate the way for future generations. We are proud to have been under your supervision, and your words and guidance will continue to light our path at every step.

To those who played a significant role in our success, whose marks were clear in our journey, and who gave us hope in the moments we most needed support. Your memories will remain in our hearts and in the cherished days of our university life, which shaped our personalities and laid the solid foundations for a bright future.

To the place that will always be etched in our memories, to the place where we lived the best days of our lives, where we learned many values and met the most wonderful people. This place will remain part of our souls, and its memories will always be a well we draw from whenever we need the warmth of the past and to see it reflected in the mirror of the present.

Abstract

With the increasing reliance on digital solutions to improve the efficiency of processes, traditional methods remain in place despite their obvious limitations. Therefore, it has become essential to adopt modern technologies that contribute to digitizing these solutions to enhance performance and expedite access to services.

This project aims to develop an innovative university application that allows students to access a range of academic and administrative services digitally.

The application enhances the student experience through a unified platform that includes various services such as checking grades, viewing class schedules, exam dates, tracking tuition fees, and submitting grievances.

In this project, modern tools and technologies were used to meet the needs of the work and help achieve the desired objectives. Flutter was relied upon to design flexible and device-compatible user interfaces, MySQL for database management, and Node.js for data processing.

The application enables students to interact with some of their academic and administrative services easily and effectively via their smartphones, facilitating access to services anytime and anywhere.

The application is expected to contribute to simplifying academic and administrative procedures, helping to save time and effort in managing university activities.

Table of Contents

List of Figures

Abbreviations

DBMS	Data Base Management System
SQL	Structured Query Language
UI	User Interface
IDE	integrated development environment
NPM	Node Package Manager
CMS	Content Management System
APLS	Application Programming Interfaces
NDB	Network Database
RDBMS	Relational Database Management System
FCM	Firebase Cloud Messaging (FCM)

Introduction

1.1 Problem Statement

Although educational institutions are moving towards digital systems, IBB University still relies on traditional systems to provide certain student services, which results in decreased efficiency and delays in accessing academic and administrative services.

The most suitable solution is to develop a digital university application that offers a specific set of student services, helping to improve efficiency and facilitate faster and more effective access to these services.

1.2 Project Objectives

The aim of this project is to develop a mobile university application that meets the needs of students, characterized by ease of use, compatibility with various devices, fast performance, enhanced interactive experience, and support for future developments. This application is ready to be used by the Faculty of Engineering and Architecture as an alternative to the traditional system for providing university services.

This project is considered the first step towards improving and developing the delivery of university services across all colleges.

1.3 Project Limitations

This work was developed using Flutter for designing the user interfaces, Node.js for data processing, and MySQL as the Database Management System (DBMS).

This project will be implemented at the Faculty of Engineering and Architecture at Ibb University, which includes several academic departments (such as the Department of Computer Science, Department of Communication, Department of Civil Engineering, and Department of Architecture). The project aims to provide a range of student services that make it easier and faster for students to access academic and administrative information.

1.4 Project Methodology

In this work, the software development lifecycle steps were followed, which include planning, requirements gathering, design, programming, documentation, testing, deployment, and maintenance. These steps were specifically applied to develop a new application that provides student services, as no previous application existed for this purpose. This application helps facilitate students' access to academic and administrative information efficiently and quickly.

1.5 Services Provided by the App

This app aims to transcend the traditional boundaries between students and their university services, making it easier and faster to access information and services with just a tap. It offers a variety of services that support the academic and administrative aspects of students, providing them with a more effective and efficient university experience.

The main services include:

1. **Digital Library:** A digital library that provides students with rich knowledge resources, opening doors to learning and education with ease and convenience.
2. **Student Grades Display:** A window that clearly reflects the student's academic performance, helping them track their academic progress accurately.
3. **Lecture Schedule Display:** An organizational tool that allows students to view their lecture schedules, helping them manage their time effectively.
4. **Exam Schedule Display:** A tool that enables students to follow their exam dates, allowing them to prepare in advance without pressure.
5. **Filing Complaints, Appeals, and Paper Requests:** An electronic platform that enables students to submit complaints and appeals easily, facilitating communication with the administration.
6. **Annual Fees Display and Status:** A financial window that allows students to monitor their financial status and pay university fees easily and conveniently.

7. **Notifications:** Instant alerts that keep students updated with the latest academic and administrative news and announcements.
8. **Assignments and Homework Reminders:** An integrated tool that reminds students of their academic responsibilities, such as assignments and homework, helping them organize their tasks.
9. **University ID and Payment Status Display:** A digital card that shows the student's administrative status, including fee payment status.

This app represents an important step towards partial digital transformation at the University of Ibb. It aims to enhance the student experience by providing innovative academic and administrative services. The app also strives to be the foundation for the development of sustainable academic applications that contribute to easing students' academic and administrative lives, improving efficiency, and providing a modern and advanced educational environment.

CHAPTER 2

Tools & Technologies Used in Development

2.1 Overview of the Tech Stack

Selecting the right technologies and tools is essential in modern application development. The chosen tech stack significantly impacts the application's performance, scalability, security, and overall user experience.

For this project, Flutter was selected for frontend development due to its cross-platform capabilities, fast performance, and rich UI components. The backend is built using Node.js, which provides high concurrency handling, and Express.js, a minimalist framework that simplifies API development. MySQL serves as the database for structured data management, while Firebase Cloud Messaging (FCM) handles real-time notifications.

This chapter provides an in-depth look at the technologies used in both the frontend and backend, explaining their purpose, features, and how they contribute to the project's goals.

2.2 Frontend Development and Technologies

The frontend of this application is built using Flutter, enabling cross-platform development with a single codebase.

This section explains the core development concepts and the technologies packages integrated into the project for achieving a seamless user experience. These technologies cover various aspects like UI development, state management, offline support, network communication, and more.

2.2.1 UI Development with Flutter

Flutter provides a widget-based UI system, allowing developers to build highly customizable, fast, and responsive interfaces.

Key Features:

- Cross-platform support for mobile, web, and desktop.
- Fast rendering using the Skia engine.
- Material & Cupertino widgets for native look and feel.

- Hot Reload for instant UI updates during development.

2.2.2 State Management with GetX

Modern mobile apps are dynamic — they continuously react to user input, backend data, and navigation changes. State management is the technique used to track and update these changes across the app. Choosing the right approach is critical for maintaining UI consistency, reducing bugs, and improving performance.

The app uses GetX, a lightweight yet powerful Flutter package that covers multiple concerns:

- Reactive state management
- Routing and navigation
- Internationalization (multi-language support)

Its simplicity, low boilerplate, and wide scope made it an ideal choice for our implementation.

2.2.3 API Communication with Dio

To interact with backend services, the app uses Dio, which provides an optimized solution for:

- Efficient asynchronous requests with automatic retries.
- Request interceptors for modifying headers, logging, or caching responses.
- Timeout handling to prevent UI freezing due to slow requests.

2.2.4 Network Awareness with Connectivity Plus

Ensuring smooth online offline transitions is essential for usability. connectivity_plus allows the app to:

- Detect internet availability (WiFi, mobile data, or no connection).
- Automatically adapt app behavior based on connection status.
- Enable offline-first strategies by handling network loss.

2.2.5 Offline Data Storage and Image Caching

Handling offline data and efficiently caching images ensures the app provides a seamless experience even in low or no network conditions. This section combines both Hive and cached network image for storage and caching.

Offline Data Storage with Hive To store data locally on the device, Hive is used as a fast, efficient NoSQL database solution:

- Fast read write operations with minimal memory usage.
- Object storage with support for Type Adapters.
- Efficient caching of API responses and user preferences, ensuring quick access to offline data.

Image Caching with cached network imageTo optimize image loading and reduce network usage, cached network image is integrated into the app:

- Caches downloaded images locally for faster subsequent access.
- Handles network failures gracefully by displaying placeholders.
- Reduces bandwidth usage by avoiding repeated downloads.

The combination of Hive for data and cached network image for images ensures that data and images are readily available, even without an internet connection.

2.2.6 File Management and Permissions

Handling user files requires secure access and permission management. The app integrates:

- file_picker for selecting files of various types.
- open_filex for opening files in external apps.
- permission_handler for requesting and managing permissions dynamically.

2.2.7 Barcode and QR Code Integration

Barcodes and QR codes are useful for authentication, product scanning, and data exchange. The app uses barcode widget, which provides:

- Support for multiple barcode formats (QR, EAN, Code128, etc.).
- Fast and customizable barcode generation.

- Lightweight integration with no extra dependencies.

2.2.8 Notifications for Real-Time Updates

The app uses notifications to keep users informed, integrating:

- firebase_messaging for push notifications, ensuring timely updates even when the app is closed.
- flutter_local_notifications for scheduled and background notifications.

2.2.9 Localization and Multi-Language Support with GetX

Supporting multiple languages improves accessibility. The app uses GetX for dynamic language switching and intl for formatting dates, times, and numbers according to locale settings:

- GetX handles language switching dynamically without restarting the app.
- intl provides locale-based date/time formatting and number formatting.

2.2.10 Time Parsing and Formatting with intl

The intl package plays a crucial role in time parsing and formatting in the app. It ensures that dates and times are displayed correctly based on the user's locale settings.

Key Features:

- Locale-based Time Formatting: intl allows the app to format dates and times based on the user's locale, ensuring they appear
- in a familiar and culturally appropriate format.
- Custom Date and Time Patterns: Developers can define specific patterns for displaying times, such as `yyyy-MM-dd HH:MM: SS`.
- Time Zone Support: intl handles time zone conversions, which is particularly useful for displaying times in different regions.
- Date and Time Parsing: It enables parsing of dates and times from strings in a variety of formats, making it easier to handle different data inputs.

2.2.11 Path Management with Path Provider

Efficient file storage requires knowing where to save files.

path_provider helps:

- Find platform-specific directories for storing files.
- Manage app-specific storage paths securely.
- Support both temporary and permanent file storage.

2.2.12 Firebase Core Setup

Firebase services enhance app functionality, requiring proper initialization. firebase_core ensures:

- Seamless integration of Firebase features.
- Background and foreground compatibility for cloud functions.
- Easy configuration across platforms.

2.2.13 Permission Management with Permission Handler

Managing permissions is essential for ensuring the app functions properly while respecting user privacy permission_handler provides:

- Requesting and checking device permissions dynamically.
- Comprehensive support for iOS and Android platforms.
- Managing permissions for sensitive features like camera, location, and file access.

2.2.14 Summary of Used Packages

Package	Purpose	Key Features
Flutter	Core framework	Cross-platform application development with a widget-based UI.
Get	State management	Lightweight state, navigation, and dependency management.
Dio	API communication	Handles network requests with advanced features like interceptors and retries.
connectivity_plus	Network status	Detects internet connectivity changes and supports offline-aware behavior.
barcode_widget	Barcode/QR generation	Supports various barcode formats for scanning and display.

Package	Purpose	Key Features
Intl	Time formatting	Formats dates, times, and numbers based on locale settings, and parses time strings.
file_picker	File selection	Allows users to pick files of various types from their device.
cached_network_image	Image handling	Efficient caching and loading of network images, reducing bandwidth usage.
path_provider	File storage	Provides access to system directories for managing file paths.
permission_handler	Permission management	Dynamically requests and checks system permissions.
Hive	Local storage	Fast, NoSQL database for lightweight and efficient data storage.
hive_flutter	Hive integration	Seamless integration of Hive with Flutter for local data access.
flutter_local_notifications	Local notifications	Manages scheduled, background, and persistent notifications.
firebase_core	Firebase setup	Initializes Firebase for cloud-based services.
firebase_messaging	Push notifications	Enables Firebase Cloud Messaging (FCM) for real-time updates.
open_filex	File opening	Opens files in external apps or native viewers.

2.3 Backend Development and Technologies

The backend of this application is built using Node.js, with Express.js as the web framework, Sequelize as the ORM, and MySQL as the database. This section explains the core backend development concepts and the integrated technologies, covering API development, authentication, security, database management, file handling, notifications, and more.

2.3.1 Backend Development with Node.js

Node.js is an asynchronous, event-driven runtime environment that allows JavaScript to be executed on the server side.

Key Features:

- **Asynchronous & Non-blocking I/O** → Ensures high performance and efficient request handling.
- **Event-driven architecture** → Ideal for real-time applications.
- **Rich package ecosystem via npm** → Provides extensive libraries for backend functionalities.

2.3.2 Web Framework with Express.js

A web framework simplifies backend development by managing request handling, authentication, and database interactions.

Express.js is a lightweight framework designed for Node.js that provides essential functionalities for building web applications and APIs efficiently.

Key Features:

- **Middleware support** → Enhances request/response handling.
- **Routing system** → Simplifies API endpoint management.
- **Easy integration** → Works seamlessly with databases, authentication, and third-party services.

2.3.3 Database Management with Sequelize & MySQL

Sequelize is an Object-Relational Mapping (ORM) library that abstracts SQL queries into JavaScript objects, making database operations simpler.

Key Features:

- Supports multiple databases (**MySQL, PostgreSQL, SQLite**).
- Handles complex relationships (**One-to-One, One-to-Many, Many-to-Many**).
- Provides built-in migration and transaction support for structured database management.

2.3.4 Authentication & Security

Request Validation with Express Validator

Ensures incoming API requests contain valid and secure data, sanitizing and validating user input before processing.

Password Hashing with bcrypt

bcrypt is a cryptographic hashing algorithm used to securely store passwords and sensitive data.

How bcrypt Works:

1. **Salting** → Generates a unique random salt to prevent attacks using precomputed hash tables.
2. **Hashing** → Repeatedly applies the bcrypt algorithm, making it resistant to brute-force attacks.
3. **Storage** → Stores only the hashed password, ensuring security.
4. **Verification** → Compares hashed passwords for authentication.

Token-Based Authentication with JWT

JSON Web Token (JWT) enables secure transmission of user session data.

How JWT Works:

1. **User Login** → A JWT is generated with user-related information.
2. **Token Signing** → The JWT is signed using a secret key.
3. **Client Storage** → The token is stored in local storage or cookies.
4. **Token Verification** → Subsequent requests include the token for authentication.

Key Benefits:

- **Stateless Authentication** → No server-side session storage.
- **Compact & Secure** → Efficient transmission with secure signing.
- **Scalability** → Suitable for microservices and distributed systems.

2.3.5 File Handling & Document Processing

File Uploads with Multer

Multer is a middleware used for handling multipart/form-data requests, particularly for file uploads.

How Multer Works:

1. **Storage Configuration** → Supports disk and memory storage options.
2. **File Filtering** → Allows filtering based on extensions, size, or other criteria.
3. **Metadata Handling** → Provides access to file metadata for validation.

PDF Processing and Image Conversion

To facilitate document previewing, the system extracts the first page of uploaded PDFs and converts it into an image.

Technologies Used:

- **PDF-lib** → Loads and manipulates PDF files.
- **PDF-Poppler** → Converts PDFs into images.
- **Sharp** → Optimizes images through resizing, compression, and format conversion.

2.3.6 Email & Notifications

Email Sending with Nodemailer

Nodemailer enables programmatic email sending using various transport methods, including **SMTP, OAuth, and third-party email services**.

Use Cases:

- Sending password reset links.

Push Notifications with Firebase Cloud Messaging (FCM)

The **Firebase Admin SDK** is used for backend integration with **Firebase Cloud Messaging (FCM)** to send push notifications to users on web and mobile platforms.

2.3.7 Task Scheduling with Node-Cron

Automating periodic tasks is essential for system efficiency. **Node-Cron** is used for scheduling operations like:

Implementation in Lecture Management System:

1. **Transaction Handling** → Ensures consistency when modifying lecture schedules.
2. **Lecture Replacement** → Marks original lectures as replaced and schedules their restoration.
3. **Restoration Scheduling** → Uses Node-Cron to revert the replacement after a defined period.

2.3.8 Data Processing & Analysis

Excel File Handling with XLSX

The **XLSX** library is used for reading and writing Excel files, supporting:

- **Data Importing** → Extracts structured data and inserts it into the database.
- **Data Exporting** → Converts database records into Excel format for reporting.

Automated Translation with Translate-Google

To support multilingual functionality, **Translate-Google** is used to translate key data fields.

Use Cases:

- **Automatic translation of user inputs**.
- **Ensuring stored data consistency across multiple languages**.

2.3.9 Utilities & Development Tools

Several utility libraries improve backend development efficiency and ensure smooth backend operations:

- **dotenv** → Loads environment variables for configuration management.
- **cors** → Enables Cross-Origin Resource Sharing (CORS) for secure frontend-backend communication.
- **nodemon** → Automatically restarts the server during development when code changes.
- **faker & @faker-js/faker** → Generates fake data for testing and database seeding.

express-validator: ensures that incoming API requests contain valid and secure data. It helps to sanitize and validate data before it is processed or stored

2.3.10 Summary of Used Packages

Package	Purpose	Key Features
Node	Core runtime environment	Asynchronous, event-driven architecture for backend development.
Express	Web framework for API development	Simplifies routing, middleware handling, and API structuring.

Package	Purpose	Key Features
express-validator	Request validation	Validates and sanitizes incoming requests to ensure security.
Sequelize	ORM for database management	Simplifies database queries, supports migrations, and table relationships.
mysql2	MySQL database driver	Enables fast and efficient MySQL database interaction.
Bcrypt	Password hashing	Securely hashes user passwords before storing them.
Jsonwebtoken	Authentication using JWT	Provides token-based authentication for API security.
Nodemailer	Email sending	Supports SMTP and third-party email providers for automated notifications.
firebase-admin	Firebase service management	Enables Firebase push notifications and other cloud functionalities.
Multer	File upload middleware	Handles multi-part file uploads securely.
pdf-lib	PDF generation	Allows creation and modification of PDF documents.
pdf-poppler	PDF conversion	Converts PDFs into image files for previews.
Sharp	Image processing	Resizes, compresses, and converts images efficiently.
node-cron	Task scheduling	Automates recurring tasks like data cleanup and email reports.
Xlsx	Excel file processing	Reads and writes Excel files for reporting and data handling.
translate-google	Google Translate API wrapper	Automates text translation into multiple languages.
Dotenv	Environment variable management	Loads configuration variables from `.env` files.
Cors	Cross-Origin Resource Sharing	Allows secure communication between frontend and backend.
Nodemon	Development tool	Automatically restarts the server on code changes.
faker & @faker-js/faker	Data generation	Generates fake user data for testing and seeding databases.

2.4 Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging (FCM) is a third-party cloud-based messaging service provided by Google. It enables applications to send and receive push .notifications across multiple platforms, including Android, iOS, and Web

FCM plays a crucial role in enhancing user engagement by delivering real-time notifications, even when the app is in the background or closed. It acts as an intermediary service that manages communication between the backend server and the client application, ensuring efficient message delivery based on device availability and network conditions. Additionally, FCM provides message queuing and automatic resending if the target device is offline at the time of message delivery.

2.4.1 Key Features of Firebase Cloud Messaging (FCM)

- .Instant Push Notifications → Provides real-time updates to users
- Cross-Platform Support → Works seamlessly across mobile and web .applications
- ,Targeted Messaging → Supports sending notifications to specific users .groups, or topic-based subscribers
- Reliable Message Delivery → Optimizes battery consumption and ensures .messages are received even under weak network conditions
- Background & Foreground Notification Handling → Manages notifications differently based on whether the app is active or running in .the background
- Custom Data Support → Allows additional payloads in messages to .trigger specific actions within the app
- Automatic Resending for Offline Devices → If a device is offline, FCM queues the message and delivers it once the device reconnects to the .internet

2.4.2 Backend Integration

To integrate FCM with the backend, the server must be configured to communicate with the FCM service by:

- Authenticating with Firebase → Ensuring the backend has the necessary .permissions to send messages
- Identifying Target Devices → Messages can be sent to individual - .devices (via FCM tokens) , user groups, or topic-based subscribers
- ,Sending Notifications → The server constructs a message with the title - .body, and optional data payload before sending it to FCM for delivery
- Handling Response & Errors → Firebase returns a response indicating - ,.whether the message was successfully delivered or if issues occurred (e.g .expired tokens, authorization errors)
- Offline Message Queueing & Resending → If a device is offline, FCM - stores the message and retries delivery once the device reconnects

2.4.3 Frontend Integration

For the frontend, FCM must be integrated into the application to receive and process notifications:

- Registering the Device → When the app is installed, it generates a unique .FCM token, which is sent to the backend for storage and future use
- Handling Notifications → Notifications are received and displayed .differently depending on whether the app is active or in the background
- Requesting User Permissions → On mobile devices, users must grant .notification permissions for the app to receive and display alerts
- Customizing Notification Behavior → Developers can control how notifications appear, whether they trigger in-app actions, or if they should .be stored for later
- Receiving Delayed Notifications → If the device is offline at the time of message delivery, it automatically receives the notification upon reconnecting to the internet

2.4.4 Notification Flow

The notification process follows these steps:

1 Backend Server

- The server prepares the notification, including the message content and recipient details.
- It sends the request to FCM, specifying whether the notification is for an individual user, a group, or topic-based subscribers.

2 FCM Service

- Firebase authenticates and processes the request.
- **Device Online**: The notification is delivered immediately to the device.
- **Device Offline**: The notification is queued and will be retried until the device reconnects to the internet (max retention period of ~28 days). If the message expires (if an expiration time is set), it will not be delivered.

3 Message Routing

- FCM determines the routing based on several factors:
  - **App State**: Foreground, Background, or Terminated.
  - **Platform**: Android, iOS, or Web.
  - **Delivery Priority**: Normal or High.
  - **Network Status**: Whether the device is online or offline.

4 Frontend Application

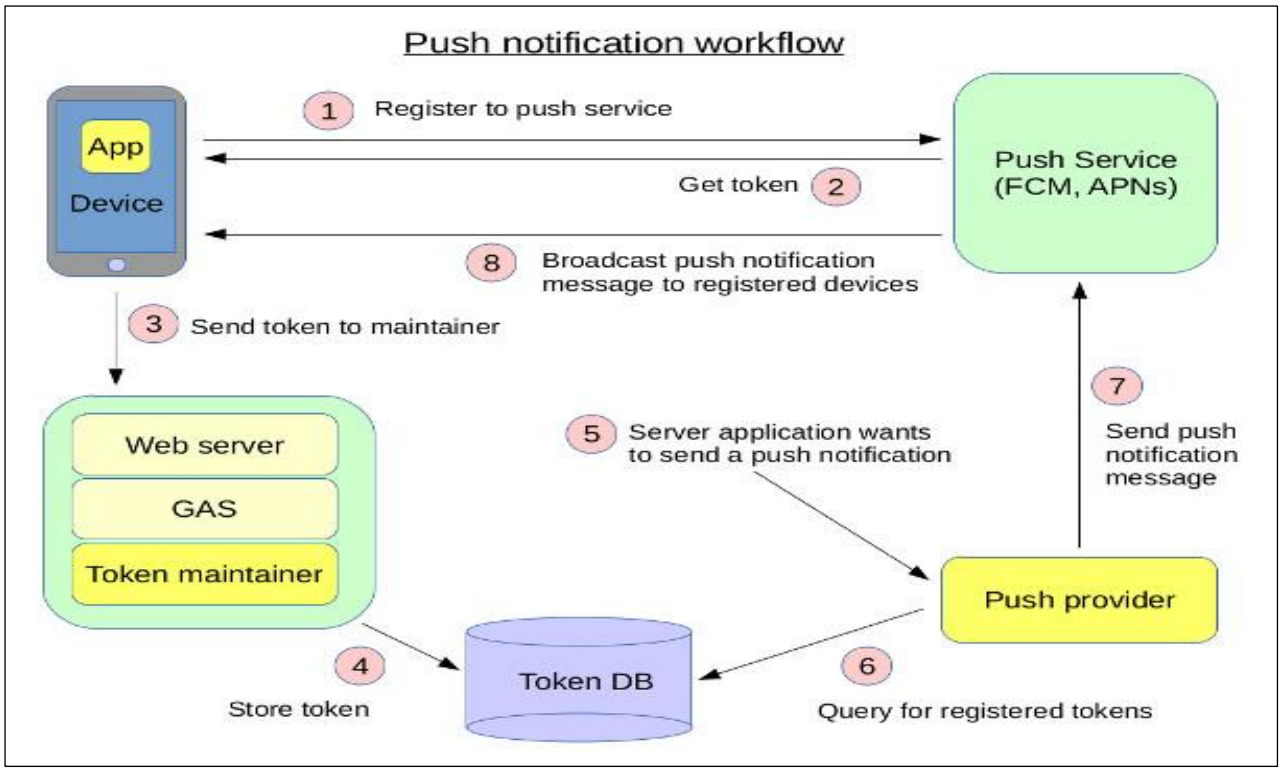
- **App in Foreground**: The app receives the notification instantly, which can trigger in-app actions such as updating the UI or fetching additional data.
- **App in Background or Closed**: The notification appears as a system alert (e.g., in the notification tray).
- **Offline Device**: The notification is queued by FCM and automatically delivered when the device reconnects to the internet.

5 User Interaction & App Response

- **User Taps Notification**: The app is opened, and the message data is passed to the app.
- The app can handle the message, update the UI, and respond accordingly (e.g., fetching more data or navigating to specific content).

2.4.5 Notification Flow Diagram

The following updated diagram illustrates how FCM stores messages when a device is offline and resends them once the device reconnects:



Figuer 2.1 Notification flow

2.5 Development Tools & Utilities

This section outlines the essential tools and utilities used in the development process to ensure smooth collaboration, effective coding, and efficient testing for both frontend and backend. These tools are fundamental for version control, API testing, managing development environments, and handling package dependencies.

2.5.1 Version Control (Git & GitHub)

Version control is crucial for tracking changes in the codebase and for collaboration. **Git** and **GitHub** are used extensively in this project to ensure that code changes are tracked efficiently and that developers can collaborate effectively.

Git: A distributed version control system that allows developers to track and manage changes in the source code. It enables:

- Efficient branching and merging of code.
- Collaboration without conflicts, as developers can work on separate branches and later merge their changes into the main branch.
- Tracking of project history and rollback to previous versions if necessary.

GitHub: A cloud-based platform for hosting Git repositories, providing a central location for developers to share and collaborate on code. Key features of GitHub include:

- Pull requests for proposing and reviewing changes.
- Issue tracking to manage tasks, bugs, and feature requests.
- Collaboration tools such as comments, code reviews, and project boards.

2.5.2 API Testing (Postman)

Postman is a comprehensive tool for testing APIs. It is used to ensure the backend services function as expected before integration with the frontend. With Postman, developers can:

- Send Custom HTTP Requests: Supports various HTTP methods such as GET, POST, PUT, DELETE, and PATCH, allowing testing of different API endpoints.
- Verify API Responses: Developers can verify if the API returns the expected response, including status codes, data, and error messages.
- Automated Testing: Postman supports automated testing, enabling the creation of test suites to ensure continuous API reliability.
- Environment Management: It allows the management of different environments (e.g., development, staging, production) for seamless testing across different configurations.

2.5.3 Development Environments (VS Code & IntelliJ IDEA)

For efficient coding, debugging, and project management, **VS Code** and **IntelliJ IDEA** are the primary development environments used in this project.

VS Code (Visual Studio Code): A lightweight, versatile code editor that supports many programming languages, including Dart and JavaScript. Features include:

- Syntax Highlighting: Makes code more readable and easier to debug.
- IntelliSense: Provides code completion, parameter info, quick info, and member lists.
- Integrated Terminal: Allows developers to run commands and scripts directly within the editor.

- Debugging Tools: Integrated debugging support for various programming languages, including Dart for Flutter.

IntelliJ IDEA: A powerful integrated development environment (IDE) used for more robust Flutter development. It supports:

- Advanced Code Intelligence: Offers sophisticated code suggestions, completions, and refactoring tools.
- Flutter Integration: Seamless integration with Flutter SDK and its tooling for live UI design and app building.
- Live UI Design Tools: Provides a visual interface for Flutter widget design.
- Robust Debugging Capabilities: IntelliJ IDEA offers comprehensive debugging support, including hot reloads for Flutter development.

2.5.4 Package Management (npm)

npm (Node Package Manager) is the default package manager for the Node.js runtime environment. It is used to manage the dependencies and libraries for the backend (Node.js and Express.js) of the project. Key features of npm include:

- Package Installation: Helps developers install project dependencies and ensures the correct versions of libraries are being used.
- Version Management: npm ensures compatibility between different package versions, avoiding issues related to outdated or incompatible libraries.
- Speeding up Setup: By managing dependencies, npm ensures quick setup and installation processes for new developers or in new environments.

2.5.5 Summary of Development Tools & Utilities

Tool	Purpose	Key Features
Git	Version control system	Tracks code changes, supports branching, merging, and collaboration.
GitHub	Cloud-based Git repository hosting	Facilitates collaboration through pull requests, code reviews, and issue tracking.
Postman	API testing tool	Allows custom HTTP requests, response verification, automated testing, and environment management.
VS Code	Code editor	Syntax highlighting, IntelliSense, integrated terminal, and debugging tools.

Tool	Purpose	Key Features
IntelliJ IDEA	Integrated development environment (IDE)	Advanced code intelligence, Flutter integration, live UI design, and debugging.
Npm	Package management tool	Installs and manages Node.js dependencies, ensuring correct versions and easy setup.

These tools are essential in ensuring smooth development processes, improving team collaboration, and maintaining high code quality. They streamline the workflow, from coding and testing to version control and deployment.

CHAPTER 3

App architecture and implementation

3.1 App Analysis and Design

This section provides a high-level overview of the system's architecture, design patterns, and key features. It explains how the app is structured and why certain design choices were made to ensure scalability, maintainability, and usability.

Note: This section focuses on concepts and design decisions. Technical implementation details, such as how caching, synchronization, authentication, and API interactions are handled, will be covered in later sections.

System Architecture Overview

The system follows a **Client-Server Architecture**, where different client interfaces communicate with the backend through **REST APIs**. This separation ensures:

- **Scalability** – The frontend and backend can evolve independently.
- **Security** – Business logic and sensitive data are managed on the server.
- **Efficiency** – The frontend handles UI and user interactions, while the backend processes data and enforces rules.

System Components

1. **Client (Mobile App & Web Page)**
   - Used by students and doctors to access services.
   - Available as both a mobile app and a web-based interface for accessibility.
   - Handles user interactions, displays data, and sends API requests to fetch and update information.
   - Supports offline access by caching key data.

2. **Admin Panel (Web - Limited to Data Management)**
   - A web-based dashboard used by administrators exclusively for data management (insert, delete, and update records).
   - Interacts with the same backend as the client interfaces via REST APIs.
   - Does not include service usage features available in the mobile/web client.

3. **Server (Backend / API)**
   - Processes requests, applies business rules, and returns responses.
   - Manages authentication and authorization.
   - Ensures data consistency and synchronization.

4. **Data Storage & Communication**
   - A relational database (MySQL) stores structured data efficiently.

- The mobile app, web client, and admin panel communicate with the backend via RESTful APIs.

Design Patterns & Principles

Using structured design patterns makes the system maintainable, scalable, and well-organized by separating concerns and reducing complexity.

Key Patterns Used

- **Model-View-Controller (MVC)** – Separates data, UI, and logic, improving maintainability. The Model represents data (e.g., users, services), the View displays it, and the Controller manages interactions.
- **Repository Pattern** – Provides a flexible data layer, making it easier to modify queries or switch databases without affecting business logic.
- **Service Layer Pattern** – Organizes business logic into dedicated services such as authentication, notifications, and data synchronization, keeping controllers lightweight and reusable.
- **Component-Based UI Design** – Encourages reusability in the frontend. Global components like buttons and cards are shared across screens, while view-specific components are customized for each page.

User Experience Considerations

Multi-Language Support

To enhance accessibility, the app supports multiple languages, specifically Arabic and English. Multi-language support is implemented at both the frontend and backend levels:

Frontend

- The UI dynamically switches languages based on the user's preference.
- Translations are stored in separate text resource files.
- Proper **right-to-left (RTL) support** is implemented for Arabic.

Backend

- **Translatable Fields:** The database uses a JSON column to store translations for multi-language fields, allowing easy retrieval of content in different languages.
- **Auto-Translation**: When new records are inserted, the system **automatically** translates content using the Google Translate API, reducing manual effort.

Offline Support

Since users may lose internet access during usage, the app ensures a smooth and uninterrupted experience through offline support mechanisms.

Caching Data

The app uses **local storage (Hive)** to cache essential data such as lectures, assignments, user details, and library files. This ensures that:

- Users can continue accessing key features even without an internet connection
- Previously loaded data is displayed instantly from local cache
- Redundant API requests are avoided, improving performance and efficiency

Caching is closely integrated with the app's repository layer, enabling automatic storage and retrieval of structured model objects.

Data Synchronization Mechanism

To keep cached data accurate and up-to-date, the system implements a custom **timestamp-based synchronization mechanism**. This strategy is designed specifically for this app, blending ideas from traditional delta sync with additional support for filtered data scopes and real-time triggers.

Backend Processing

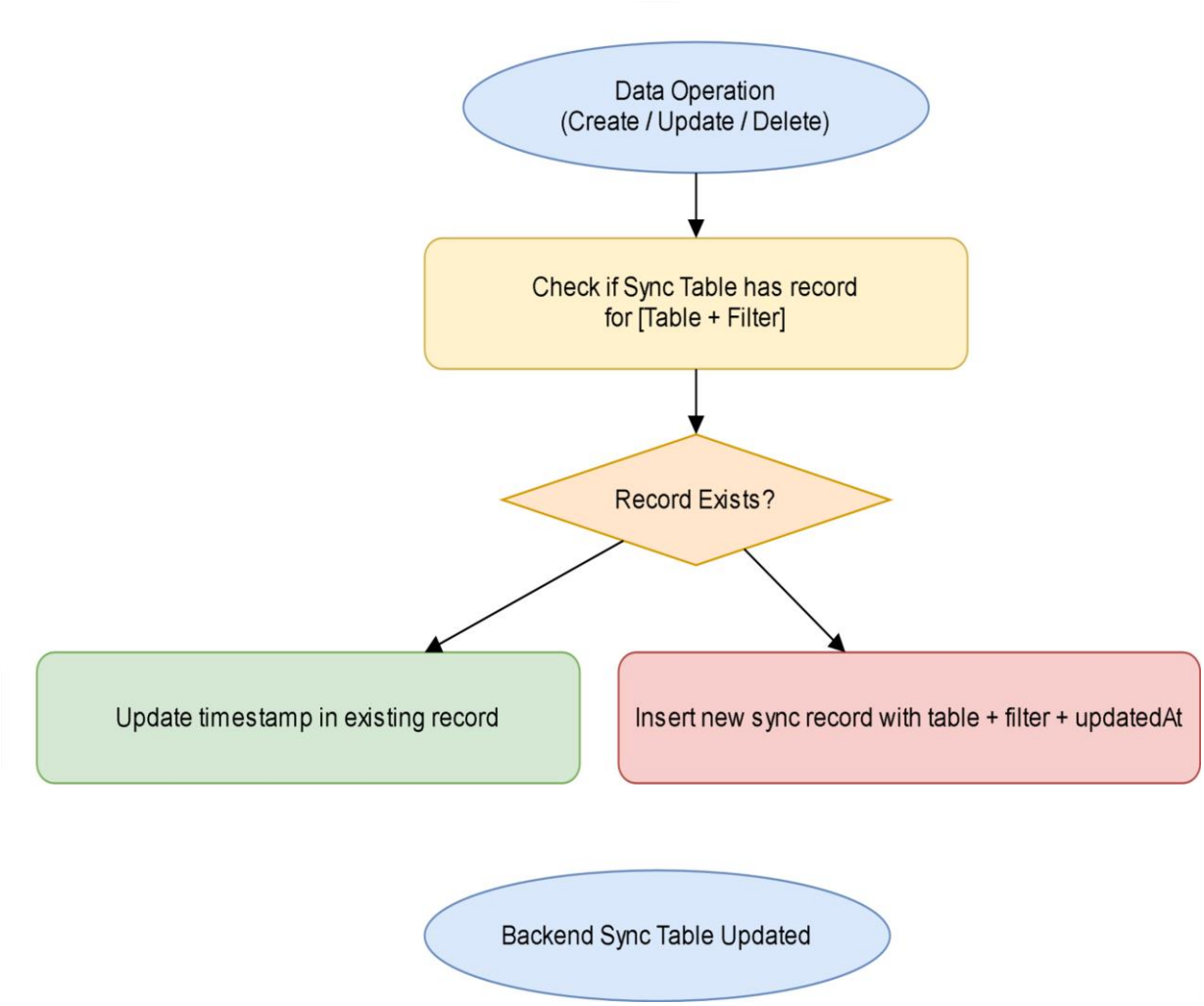
The backend maintains a dedicated **Sync Table** that tracks:

- The last updatedAt timestamp for each table
- Optional **filters stored as JSON** (e.g., level ID, section ID), allowing different states for subsets of data

Whenever a data operation occurs (create, update, delete), the backend either:

- Updates the timestamp of an existing sync record for that table and filter
- Or inserts a new record if no matching state exists

This enables precise detection of what data has changed and under which filter.

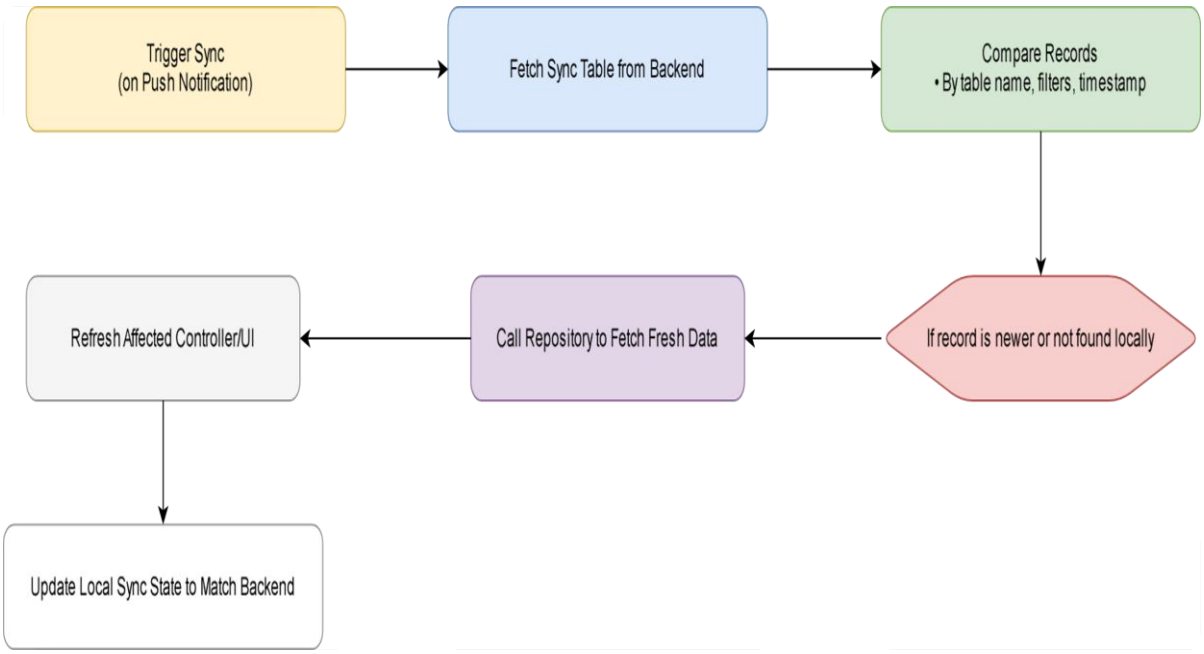


Frontend Synchronization Flow

The frontend keeps a local copy of the last known sync state. The sync process is triggered when a push notification (via FCM) is received

The sync service performs the following steps:

1. **Fetch the latest Sync Table** from the backend
2. For each record, **compare timestamps and filters** against the local copy
3. If a newer timestamp is found (or if the record didn't exist locally), trigger a **targeted data refresh** via the relevant repository
4. After all updates are processed, the local sync table is updated to match the backend



This approach ensures that:

- Cached data stays consistent without requiring full refreshes
- Data sync is context-aware (e.g., syncing only relevant records for the user)
- Users experience a responsive and seamless app — both online and offline

Security & Performance Considerations

To ensure system reliability and protect user data, the app incorporates various security and performance optimizations:

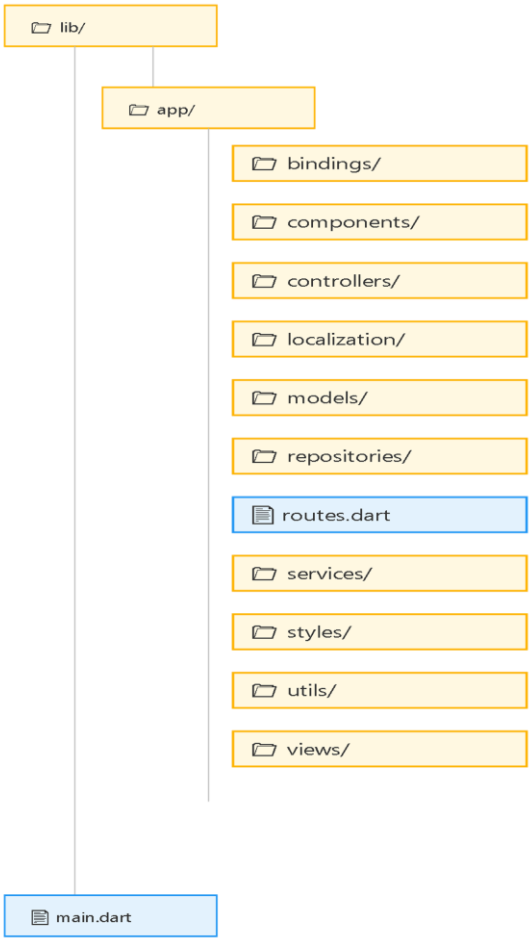
- **JWT Authentication** – Users authenticate using JSON Web Tokens (JWT), which secure API requests.
- **Role-Based Access Control (RBAC**) – User permissions vary depending on their role (e.g., student, doctor, admin).
- **Local Storage Caching** – The frontend caches data locally for faster loading times and offline data view.

3.2 Frontend Implementation

This section presents how the frontend of the app was structured and built using Flutter. It focuses on architectural decisions, tools, and the connection between UI components and logic, without going into full development details.

1. Project Structure

As mentioned earlier, we follow the **MVC (Model-View-Controller)** pattern, which is a great way to keep everything organized. It helps us separate the app's logic, UI components, and data management, making it easier to scale and maintain over time. Let's dive into how the project is structured:



Folder Breakdown: What Each Folder Does

app/

This is where the core of the app resides, containing everything that drives the app's functionality. Here's what each folder in app/ is responsible for:

- **bindings/**:
  Handles the **setup and dependency injection** for the app. This ensures each component gets what it needs to function properly.
- **components/**:
  Contains **reusable UI elements** that appear across multiple screens, such as buttons, cards, and custom widgets. This promotes consistency and avoids duplication of code.
- **controllers/**:
  Houses the **business logic** of the app. These controllers manage the app's

state, fetch data from the backend, and update the UI as necessary. We use **GetX** for reactive state management, which ensures the UI updates automatically when the data changes.

- **localization/**:
  Manages **language support**, enabling the app to be used in different languages.

It contains translation files and Getx localization setup class

- **models/**:
  Defines the **data structures** used throughout the app. These models represent the data (e.g., user details, lecture information) and allow us to easily parse and manipulate it.
- **repositories/**:
  Responsible for **data fetching** and **caching**. Repositories handle the app's communication with APIs or databases and cache the data for future use.
- **services/**:
  Contains code for integrating with **external services** like APIs, Firebase, and storage solutions. These services help the app communicate with the outside world.
- **styles/**:
  Holds the app's **styling and themes**, such as colors, fonts, and layout settings, ensuring a consistent look and feel throughout the app.
- **utils/**:
  Contains **helper functions** for common tasks, like formatting dates or validating input. These functions are used across the app to simplify repetitive code.
- **views/**:
  Houses the **UI screens** for different pages of the app. Each screen is defined here, where we specify how the UI should be displayed, including layouts, components, and user interactions.
- **routes.dart**:
  Manages **app navigation** by defining named routes. It makes navigating between screens easier and more consistent.

main.dart

This is the **entry point** of the app. It initializes the app's configuration (like themes, routes, and services) before the app starts running.

Key Principles Behind This Structure

- **Separation of Concerns**: We keep the UI, business logic, and data management in separate areas to make the app easier to work with. This structure is clean and ensures that developers can focus on one part of the app at a time without getting overwhelmed.

- **Feature-Based Structure**: Each feature (like login, profile management, or lectures) has its own dedicated folder, which makes it easy to scale the app as new features are added.
- **Reusability:** By creating reusable components, we can avoid repeating code. If something is used on multiple screens, we put it in the components folder and just reference it when needed.

2. GetX Role in Frontend Implementation

As previously mentioned in **Chapter 2: Technology and Tools,** the app uses **GetX** as the main state management and routing solution due to its lightweight nature, reactive programming support, and built-in tools for navigation, dependency injection, and localization.

In this chapter, we focus on how GetX was applied practically throughout the frontend implementation.

GetX played a key role in multiple layers of the app, including:

- State and logic management through reactive controllers
- Modular screen routing
- Centralized service injection
- UI reactivity and responsiveness
- Localization handling
- Utility access for screen sizing and messaging

These features contributed to building a clean, organized, and scalable frontend structure.

Core GetX Components Used in the App

Below are the key GetX components integrated into the app, along with their purpose and how they contribute to the app:

App Foundation

- GetMaterialApp
  Acts as the backbone of the entire app. It replaces the regular MaterialApp and enables all of GetX's features — routing, state management, localization, snackbars, dialogs, and more — with zero extra setup.

  Used in: main.dart as the root widget to initialize the app.

Controllers and State Management

- GetxController
  Every major screen has a dedicated controller that manages its state and logic, such as handling API calls, updating values, and reacting to user actions.

  Example: LoginController manages input validation and calls UserRepository.login() when the login button is pressed.

- GetView <T>
  A convenience class used in screen widgets to easily access their controller without writing extra boilerplate code.
- Obx
  The most frequently used widget for observing reactive (Rx) variables. Any change in data automatically updates the UI without manual triggers.
- GetBuilder
  Used in a few places where performance matters, allowing manual rebuilds using update(). This is helpful for situations where reactivity doesn't need to be continuous.

Dependency Injection and Modularity

- **Bindings**
  Each screen has an associated binding class, responsible for injecting its dependencies. This keeps screen setup modular and clean, especially for large apps.
- **Get.put () / Get.lazyPut()**
  Used to inject controllers and services into memory:

  - Get.put () immediately creates and stores the instance.
  - Get.lazyPut() delays the creation until it's first used.

  Controllers are usually registered through bindings using lazyPut () for efficiency.

Routing and Navigation

- **GetPage**
  Used to declare all the app's named routes. It connects each screen with its binding and helps in maintaining centralized route management.

- Navigation methods: Get.to (), Get.off ()
  These methods simplify navigating between screens, without needing to use context.
  - Get.to () → push a new page
  - Get.off () → replace the current page

Used in flows like login → home, or navigating from subject list to details.

Localization & Language Handling

- **.tr and Get.updateLocale()**
  GetX provides built-in support for internationalization. All display strings are stored in translation maps and accessed using (.tr).
  The app supports Arabic and English, including proper RTL layout support.

Feedback and UI Helpers

- **Get.snackbar()**
  Used to give quick feedback to users for actions like login success, form errors, or saving data.
- **Get.dialog()**
  Used to display custom popup cards, such as forms for adding or editing data, without needing context. It simplifies dialog handling and helps keep UI interactions clean and consistent.

Example: When a user taps the "Add" button, a styled popup card appears using Get.dialog() to collect input.

- **Get.width / Get.height**
  Helpful utilities to get screen dimensions directly, without Media Query, allowing cleaner responsive layouts.

This modular and consistent use of GetX helped simplify logic, reduce boilerplate, and maintain a clean separation between UI and business logic throughout the app.

3. Service & Utility Classes

To maintain a clean and modular architecture, the app uses a set of service and utility classes. These classes handle core functionality such as API communication, authentication, notifications, and localization—keeping logic out of UI components and controllers for better separation of concerns.

This section outlines the main service and utility classes used throughout the app, explaining their responsibilities and how they contribute to the overall structure.

- **HTTP Provider class**

The HttpProvider class acts as the core communication layer between the app and the backend API. It's built on top of Dio, a powerful HTTP client for Dart, and provides a centralized, consistent approach to handle all HTTP requests across the app.

Key Responsibilities:

- Manages all standard HTTP methods: GET, POST, PUT, and DELETE.
- attaches headers to requests (e.g., JWT authentication tokens).
- Stores and updates the authentication token securely.
- Handles file uploads and downloads, including multipart/form-data and streaming handling.
- Centralizes error handling by catching exceptions and returning consistent responses.

This design helps isolate networking logic, making it reusable, testable, and easy to maintain, while giving calling layers full control over how they process the received data.

- **Notification Service class**

The notification service integrates with Firebase Cloud Messaging (FCM) to manage push notifications.

Responsibilities:

- Initializes FCM and listens for new messages.
- Displays notifications while the app is in the foreground.
- Handles redirection or actions based on notification data.
- Prepares the app for background and terminated-state message handling.

This ensures real-time updates and enhances communication with users.

- **HiveServices Classes**

This class manages local data storage using the Hive database. It registers all required adapters for custom models and handles opening, closing, and clearing Hive boxes.

Key Responsibilities:

- Registers all model adapters used across the app.
- Opens global data boxes on app startup (e.g., lectures, subjects, users).
- Provides methods to clear or close boxes when needed.

It plays a key role in enabling offline access by storing frequently used data locally.

- **DataSyncService class**

The DataSyncService class is a central utility responsible for handling **frontend data synchronization** when changes are detected on the backend. It ensures that only updated data is re-fetched from the server and that the app's cache stays consistent.

This class works closely with the backend's sync tracking mechanism and plays a key role in the app's offline support and performance optimization.

Key Responsibilities

- Compare the current sync state with the last locally cached version.
- Refresh data only if newer updates exist.
- Trigger the appropriate repository methods to refetch and update local cache.
- Notify relevant UI controllers (if active) to refresh their state after sync.

4. Utility Classes

A number of utility classes are defined to support common tasks across the application. These are small, focused classes that encapsulate specific helpers and tools.

Below is a description of each:

1. dobule_digits_parse.dart

This utility ensures that floating-point numbers are displayed with a consistent number of digits, often formatting values to include a leading zero or fixed decimal places when necessary.
It's especially useful for creating uniform visual representations in areas like scores, prices, or measurements.

2. internet_connection_cheker.dart

Handles internet availability detection, helping the app respond to online/offline states (e.g., show a message, block a request, cache offline data).

3. local_lisenter.dart

wrap app local on observable variable and handle Local update globally.

4. responsivity.dart

Currently includes a single but important function: fontScale (), which adjusts text sizes based on screen dimensions or scaling logic — helps maintain visual consistency across devices.

5. snake_bar.dart

Wraps GetX's Get.snackbar() functionality in a cleaner helper function for uniform usage throughout the app. Likely offers styling presets or reusable logic (e.g., success/error messages).

6. date_time_utils.dart

Formats and parses date/time strings, durations, timestamps — typically used in UI and reporting.

7. file_utils.dart

Manages file-related tasks such as opening, writing, deleting, files.

8. json_utils.dart

Handles safe JSON parsing for encoding json content.

9. maping_data.dart

Facilitates the transformation or mapping of raw, relational, or numeric data into a display-friendly format. This is especially useful when backend data needs to be converted into human-readable forms, such as mapping status codes, category IDs, into user-friendly values being view in screen.

10. screen_utils.dart

Provides screen type determines function to get target devise view (web or mobile).

11. validators.dart

A set of reusable input validation methods (e.g., email format, required field, password strength), used in forms across the app.

5. Authentication & Authorization on Client Side (Front-End)

On the client side, authentication and authorization are key for securing access to different parts of the app. The app uses JWT (JSON Web Token) for authentication and role-based for authorization.

1. **Authentication:**
    1. **JWT Token**: Upon successful login, the backend issues a JWT token, which is securely stored in HiveBox.
    2. **Token Validation:** Every time the app makes an API call, the JWT token is included in the request headers. The server validates the token before processing the request.
    3. **Logout:** To log out, the token is removed from HiveBox, and the user is redirected to the login screen.
2. **Authorization:**
    1. **Role-based Access:** The app uses role-based authorization to manage user permissions. The backend sends the user's role (e.g., admin, student) along with the user data.
    2. **Front-End Role Handling:** Depending on the user's role, the front-end will show or hide certain elements or features (e.g., only admins can access certain settings like add).
    3. **Authorization Check:** The front-end checks for the user's role before view screens. If the role is unauthorized some elements will hide like add button

6. Repositories and Data Access Layer

Repositories are responsible for **organizing and abstracting all data-related logic** in the app. They act as a bridge between the backend API, local storage, and the controllers.

Each feature module (such as lectures, users, or assignments) has a corresponding repository that centralizes all operations for that type of data.

Key Responsibilities

- Handle **retrieval**, **creation**, **updating**, and **deletion** of data
- Serve as the single access point for each data type (e.g., subjects, exams)
- Support both **online API communication** and **offline Hive caching**
- **Automatically cache** responses after fetching from the server
- **Automatically check cache** before making a request (unless forceFetch is true)
- Simplify how the rest of the app accesses data (controllers don't need to handle cache or fetch logic manually)

7. Models and Data Structure

Models are the backbone of the app's data structure on the frontend. Each backend entity has a corresponding Dart class that defines how the data is represented, used, and passed around within the app.

Key Responsibilities

- **Structure**: Models clearly define the shape and fields of each data entity.
- **Hive Adaptivity**: Models are annotated with Hive type adapters, allowing them to be saved and retrieved directly using Hive's local NoSQL storage.
- **Consistency**: They ensure a consistent, typed structure when sending and receiving data from the backend, enabling safer and more readable code.
- **Reusability**: Models are reused across views, controllers, and repositories for both API parsing and local caching.

in this app architecture**,** repositories handle the actual saving and fetching of model objects**,** while models define the schema that represent data on the app and allows Hive to serialize/deserialize the data automatically**.**

Backend-to-Frontend Mapping

For every main entity on the backend (e.g., Student, Subject, Assignment), there is a **corresponding model** on the frontend. This 1:1 mapping ensures tight synchronization between frontend and backend structures, making data handling easier and more predictable.

Multi-language Fields

Some models contain fields that need to support multiple languages. Instead of duplicating fields for every language, these models use a **map-based structure**:

```
@HiveField(1)
Map<String, dynamic>? nameData;
```

A custom **getter** is used to fetch the correct value based on the currently selected language:

```
String? get name {
  String currentLang = Get.locale?.languageCode.toString()??"en";
  return nameData?[currentLang];
}
```

This keeps the models compact and easily adaptable to language changes without additional transformation layers.

8. Controller Structure & Usage in Screens

Controllers in the app serve as the **central layer for managing screen-specific logic and reactive state**. Each controller acts as the **logic brain** behind a screen or module, keeping UI code clean and separating concerns.

The app uses **GetX's** GetxController as the base for all controllers, which provides lifecycle hooks, reactivity, and tight integration with bindings and services.

Key Responsibilities of Controllers

- Manage screen-level data and state (e.g., form input, loading indicators)
- Validate user input and interact with form elements
- Connect to repositories or services for data operations
- Trigger navigation and localization changes
- Expose Rx variables that the UI observes using Obx or GetBuilder
- Optionally provide dynamic UI elements like dropdown options or labels used across widgets

While controllers do not contain full UI layouts, they **may include UI-related elements** that need to be controlled or updated dynamically (e.g., list of DropdownMenuItems, tabs, chips, etc.).

How Controllers Are Integrated

Controllers are tightly integrated into the app through **GetX's binding system**, which manages dependency injection and lifecycle behavior automatically.

- **Created per screen or feature module** to keep logic self-contained and modular.
- **Registered through bindings** using both Get.put () and Get.lazyPut(), depending on the screen's usage pattern:
  - Get.lazyPut() is used for **tab-based screens**, where the same controller may be revisited multiple times through navigation. This approach delays initialization until the controller is first used, improving efficiency for screens that remain in memory.
  - Get.put () is used for **standalone screens with separate routes**, such as login or detail pages that are accessed through navigation. This ensures the controller is fully initialized when the screen is pushed.
- Accessed in views via:
  - GetView<T> — which simplifies direct access to the controller inside a widget class.
  - Get.find<T> () — used for manual retrieval when needed.
- Fully **lifecycle-aware**, using onInit (), onReady (), and onClose () to:
  - Initialize state or load data

- Perform logic once the UI is ready
- Dispose resources like TextEditingControllers or FocusNodes

This integration allows the app to remain clean and reactive, while maintaining strong separation between **UI** and **logic**, and providing fine-grained control over controller instantiation and performance.

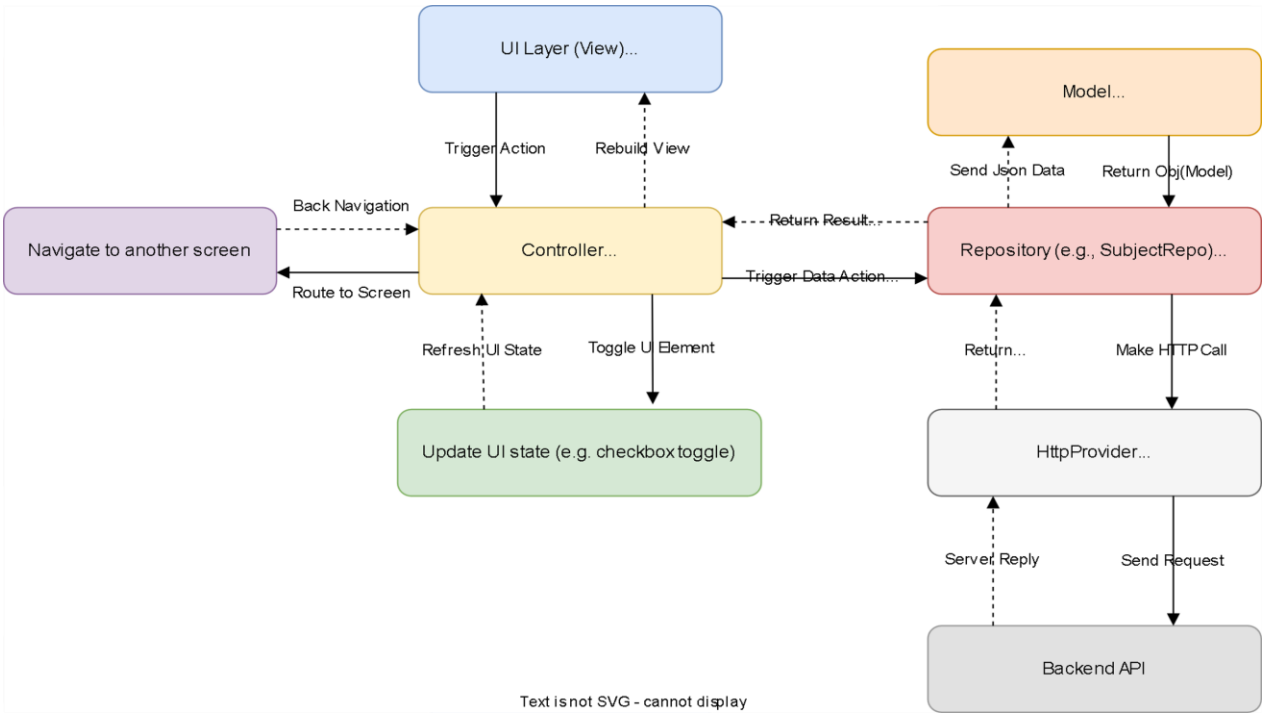
9. Frontend Data Flow and Communication

This section presents a generalized view of how the frontend layers communicate and coordinate during any action — whether it's navigating, toggling a setting, or accessing backend data.

Rather than focusing on specific business logic, this overview shows how **responsibilities are distributed** and how data or instructions flow between components.

Layered Process Flow Overview

The following diagram shows the communication and decision flow between frontend layers. It applies to all user-triggered events, regardless of the specific screen or feature.



Description of the Flow (Narrative Form)

1. A **user interacts with the UI** — for example, taps a button or submits a form.
2. The **controller receives the event**:
    - If it's a **local action** (e.g., checkbox toggle), it directly updates state.
    - If it's a **navigation** event, it triggers route change (e.g., Get.toNamed()).
    - If it's a **data access** event, it delegates the work to the corresponding repository.
3. The **repository function executes**, often cross-bonded to a backend entity.
    - It prepares the correct API URL and method.
    - Passes them to the HttpProvider to handle the request.
4. HttpProvider makes the request:
    - Adds headers and token
    - Waits for the response
    - Wraps the result in a standard Result object
5. The **repository receives the response**:
    - Parses it into the appropriate model(s)
    - Caches it locally (Hive) if applicable
    - Returns the result (success or failure) to the controller
6. The **controller updates UI state** (e.g., RxList or RxBool)
7. The **UI rebuilds automatically** in response to updated reactive variables via Obx () or GetBuilder ()

10. UI & Screens (View) and User Interaction

The View Layer is the part of the application responsible for presenting data and receiving user input. It is located under the app/views/ directory and structured around feature-based modules. Views are built using a **component-based architecture** to ensure reusability and visual consistency.

Each screen is linked to its logic using GetView<T>, a GetX utility that binds the view to its controller. Reactive UI updates are managed through Obx () and Rx variables inside each controller. Views do not contain business logic — instead, they simply display the interface and call methods from the controller when interaction occurs.

View Layer Structure

- Views extend GetView<Controller> and directly access controller logic without boilerplate.
- Reactive state changes (e.g., loading, toggles) are handled using Obx () widgets.
- Layouts follow a **clean separation of presentation and logic**, using component wrappers and utility classes.

Global & Base Components

To promote reusability and maintain design consistency, the app defines several **custom components** shared across different screens. These components abstract commonly used UI patterns into configurable widgets.

CustomButton

A reusable button wrapper around ElevatedButton, styled according to the app theme.

Features:

- Handles **default, pressed, and disabled states**
- Supports **icons and labels**
- Allows customization of **color, size, and border radius**
- Can be easily paired with Obx () for reactive loading states

CustomText

A simplified wrapper for Text that:

- Defaults to the app's global style (AppTextStyles.mainStyle())
- Supports maxLines, TextAlign, and TextOverflow.ellipsis
- Used throughout the UI for labels, headers, and dynamic content

CustomTextFormField

An enhanced input field that supports both **standard** and **password** entry modes.

Features:

- Built-in **form validation support**
- Reactive password visibility toggle using Obx ()
- Configurable input types, icons, and border styling
- Connects to TextEditingController and integrates with Form validation flow

TypeAhead

A custom widget combining **searchable dropdown** and **text input** functionality.

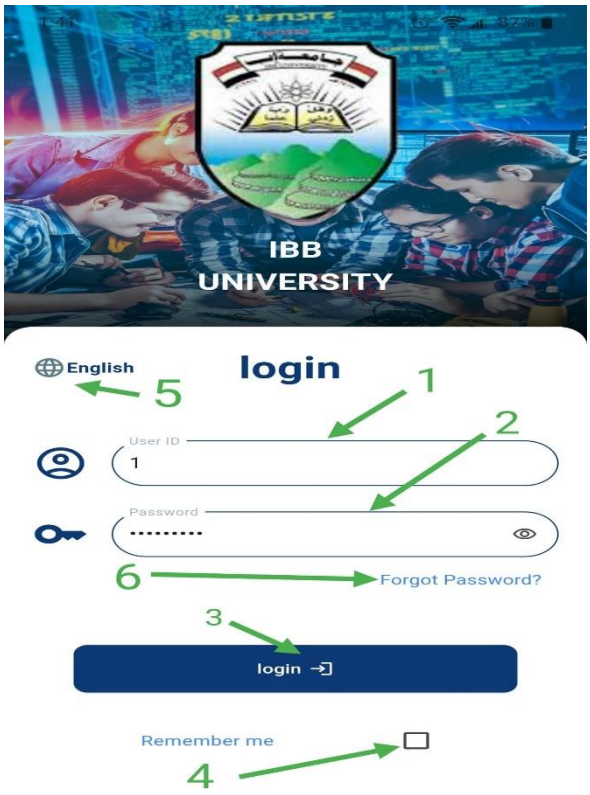
Features:

- Displays a filtered overlay menu based on user input
- Accepts Map<T, String> as its data source
- Customizable text styles, height, width, and suffix icon
- Supports an optional "All" selector
- Ideal for filtered selections like **subjects**, **semesters**, or **roles**

App Screen Breakdown

Login Screen

Purpose: Allows users to log in with their university credentials and switch between supported languages.



Component Interactions

1. ID Field
Text input controlled by LoginController.id. Captures the user's university ID and is validated on login.

2. Password Field
Secure input tied to LoginController.password. Hidden by default, with visibility toggle.

3. Login Button
Triggers LoginController.onLogin():

- On success → navigates to "/main" via Get.offNamed
- On failure → shows a Get.snackbar() with an error message

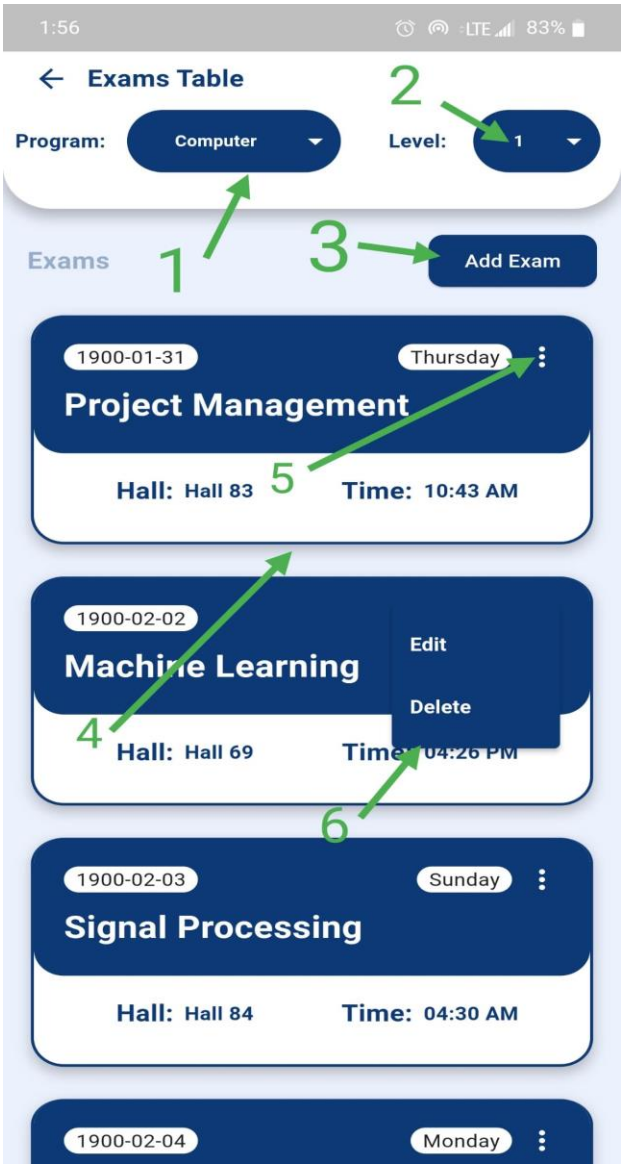
4. Remember Me Checkbox
Binds to rememberMe in the controller. When enabled, stores login state locally using Hive.

5. Language Switch
Changes the app language via changeLang(lang) in the controller. Uses LocaleListener.updateLocale() and updates the UI instantly through GetX.

6. Forgot Password Link
Navigates to /forgotPassword using Get.toNamed().

Exams Table Screen

Purpose :Displays a list of exams based on the selected program and level. Users can manage exams (add, edit, delete) through a clean, filterable interface.



Component Breakdown

1. Program Filter

Dropdown menu allowing the user to select a program (e.g., "Computer").

Triggers a fetch for relevant exams when changed

2. Level Filter

Dropdown next to the program selector used to specify the academic level.

Combined with program filter to scope results.

3. Add Exam Button

Initiates a form popup via Get.dialog() to add a new exam.

Used primarily by admins or authorized users.

4. Exam Cards

Each card shows subject name, date, time, and hall.

Cards are generated based on filtered results.

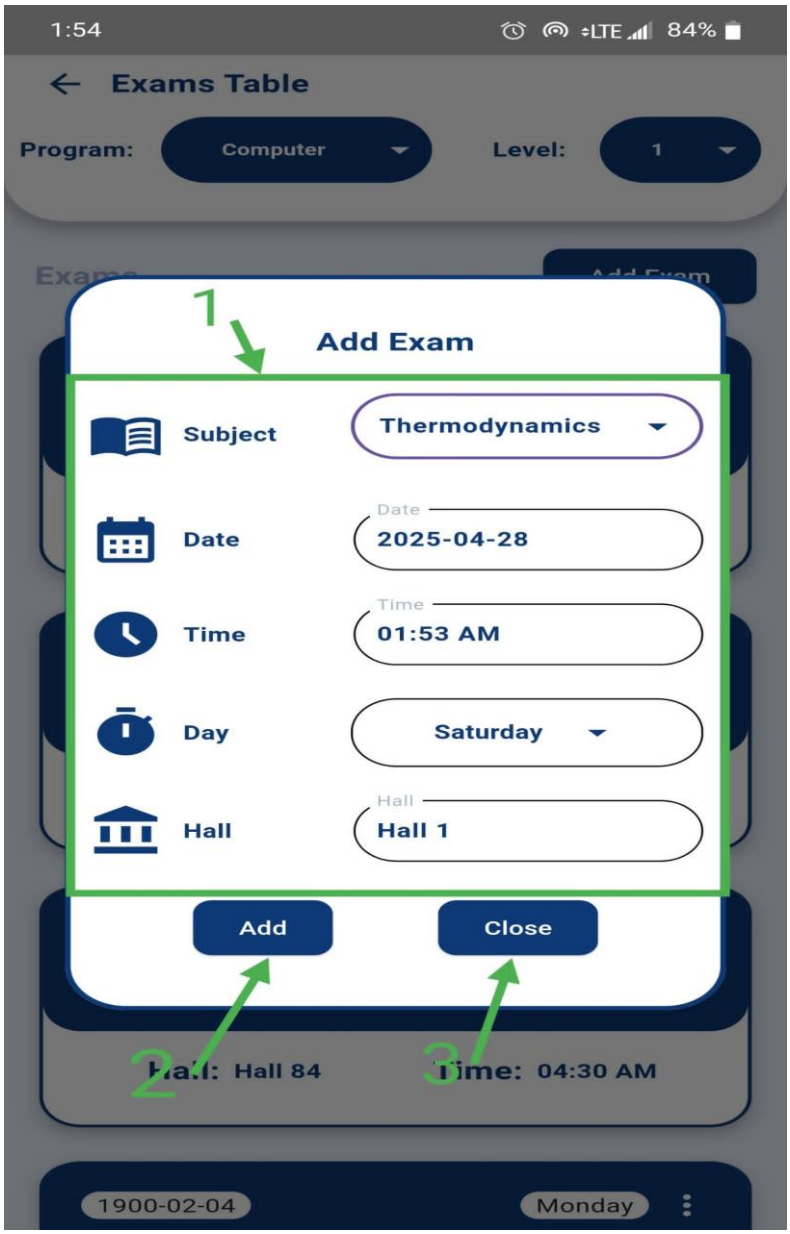
5. More Options Button (⋮)

Appears in the top-right corner of each exam card.

When tapped, reveals additional actions (edit/delete).

6. Popup Menu Actions

- **Edit**: Opens the exam in a form for modification
- **Delete**: Removes the exam from backend and cache after confirmation



Add/Edit Exam Dialog

- Triggered by:
  - **Add Exam Button** (to create a new record)
  - **Edit option** in the popup menu (to modify an existing one)
- Opens a form inside a custom Get.dialog() popup.
- Fields are pre-filled when editing.
- Submits data via ExamsController.createExam() or.updateExam().

Backend Implementation and Architecture

This section describes how the backend of the application was structured and implemented, covering project organization, database design, API request handling, and usage of key frameworks and libraries.

The backend is built using Node.js and Express.js for handling HTTP requests, with Sequelize ORM managing communication with a relational database. The architecture follows a layered, modular design, promoting separation of concerns, scalability, and maintainability.

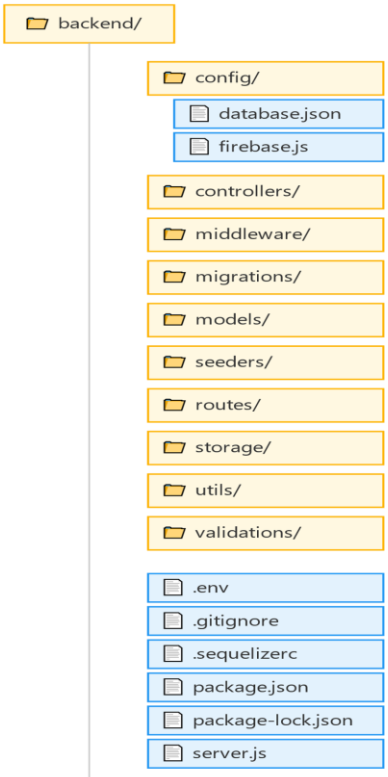
The system is organized into several key parts:

- A clear and consistent project file structure.
- A well-defined database schema design, representing the system's core entities and relationships.
- A standardized API layer handling authentication, authorization, validation, business logic, and response formatting.
- Sequelize ORM integration to manage models, migrations, and database interactions.

The following sections provide a detailed breakdown of these areas.

1. Project File Structure

The backend follows a modular project organization where each folder has a clearly defined role, making the codebase easier to maintain and scale over time



Backend Folder Breakdown:

This section outlines the roles of each folder in the backend codebase.

config/

Centralized Configuration Management

- Contains environment-specific settings and third-party service configurations like Firebase and Sequelize.

controllers/

Business Logic Handlers

- Handles HTTP requests, processes business logic, and calls models or services before sending responses.

middleware/

Request Pre-processing Layer

- Manages authentication, role-based access control, and translation.

migrations/

Database Schema Versioning

- Maintains a record of all database structure changes over time for collaborative development.

models/

Data Representation Layer

- Defines tables, relationships, and constraints used by Sequelize ORM.

seeders/

- Seed data for development/testing.

routes/

API Endpoint Mapping

- Connects HTTP requests to corresponding controller functions in a structured way.

storage/

File Handling System

- Stores and manages uploaded files like Books, Assignments, or images

utils/

Utility Functions

- Reusable helper functions used across multiple components to avoid code duplication

validations/

Input Validation Layer

- Defines rules for validating request data using libraries like express-validator.

.env

- Stores environment variables (configuration secrets) that should not be hardcoded or committed to version control.

.gitignore

- Specifies files/folders that Git should not track (e.g., dependencies, logs, or secrets).

.sequelizerc

- Configuration file for Sequelize (ORM) to customize paths for migrations, seeders, and models.

package-lock.json

- Automatically generated by npm to lock dependency versions exactly, ensuring consistent installs across environments.
- Prevents version conflicts (critical for production stability).

package.json

- ❖ Defines project metadata, scripts, and dependencies.

server.js

- ❖ Entry point of the backend application. Initializes the server, middleware, and routes.

Key Principles Behind This Structure

- ❖ **Separation of Concerns**

Each component has a well-defined responsibility to promote clarity and focus.

- ❖ **Consistency**

A uniform folder structure ensures smooth onboarding and collaborative coding.

- ❖ **Extensibility**

Modular design allows for easy feature expansion without disrupting existing code.

- **Maintainability**

Organized logic and centralized configuration make debugging and upgrades easier.

2. Database Schema Design

The database serves as the foundation for data storage, security, and system relationships.
It is designed to represent the major entities and interactions needed for the application while ensuring referential integrity and scalability.

Sequelize ORM is used to define models corresponding to database tables and manage database migrations. This abstraction simplifies database operations, maintains consistency, and supports easier collaboration across development environments.

The database schema includes core tables such as Users, Roles, Permissions, Exams, Grades, and others, each with defined relationships such as one-to-many and many-to-many associations.

The following subsections outline the database tables, their fields, relationships, and how Sequelize ORM is utilized.

3. Tables Design and Relationships

1. Section Table

The section table represents sections within a specific department or course, and it includes multiple relationships to other tables.

This helps organize the data and manage associations related to lectures, assignments, exams, and more.

Database Schema Design

The section table is crucial for managing academic sections.

The section table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
section_name	JSON	The name of the section (stored as a JSON object for multiple languages).

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established:

1.One-to-Many Relationship (Section - Study Plan Element)

Each section can have many study plan elements.

Implemented via :

section.hasMany(models.study_plan_element, { foreignKey: 'section_id' }).

2.One-to-Many Relationship (Section - Student Fee)

Each section can have multiple student fee records.

Implemented via :

section.hasMany(models.student_fee, { foreignKey: 'section_id' }).

3.One-to-Many Relationship (Section - Lecture)

A section can have multiple lectures.

Implemented via :

section.hasMany(models.lecture, { foreignKey: 'lecture_section_id' }).

4.One-to-Many Relationship (Section - Exam)

Each section can have many exams linked to it.

Implemented via :

section.hasMany(models.exam, { foreignKey: 'exam_section_id' }).

5.One-to-Many Relationship (Section - Grade)

Multiple grades can be associated with each section.

Implemented via :

section.hasMany(models.grade, { foreignKey: 'section_id' }).

6.One-to-Many Relationship (Section - User)

Each section can have many users (students, professors, etc.).

Implemented via :

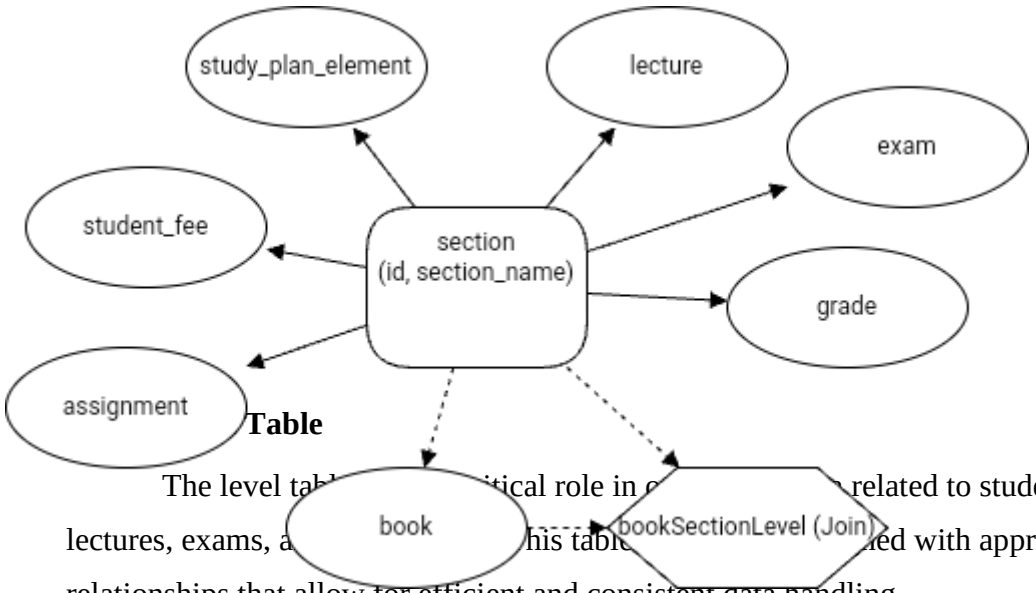
section.hasMany(models.user, { foreignKey: 'user_section_id' }).

7.Many-to-Many Relationship (Section - Book)

A section can have many books through the bookSectionLevel table.

Implemented via :

section.belongsToMany(models.book, { through: 'bookSectionLevel', foreignKey: 'sectionId' })



Table

The level ta itical role in c related to students, fees, lectures, exams, a his tabl ed with appropriate relationships that allow for efficient and consistent data handling.

Database Schema Design

The level table stores information related to the academic levels in which students are placed.

The level table includes the following columns

Column Name	Data Type	Description
id	INTEGER	(PK)Unique identifier for each level.
level_name	INTEGER	Name or identifier of the academic level.

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established

1.One-to-Many Relationship (Level - Student)

A level can have multiple students associated with it.

Implemented via :

level.hasMany(models.student, { foreignKey: 'student_level_id' }).

2.One-to-Many Relationship (Level - Study Plan Element)

A level can have multiple study plan elements associated with it.

Implemented via :

level.hasMany(models.study_plan_element, { foreignKey: 'level_id' }).

3.One-to-Many Relationship (Level - Student Fee)

A level can have multiple student fees associated with it.

Implemented via :

level.hasMany(models.student_fee, { foreignKey: 'level_fees_id' }).

4.One-to-Many Relationship (Level - Lecture)

A level can have multiple lectures associated with it.

Implemented via :

level.hasMany(models.lecture, { foreignKey: 'lecture_level_id' }).

5.One-to-Many Relationship (Level - Exam)

A level can have multiple exams associated with it.

Implemented via :

level.hasMany(models.exam, { foreignKey: 'exam_level_id' }).

6.One-to-Many Relationship (Level - Grade)

A level can have multiple grades associated with it.

Implemented via :

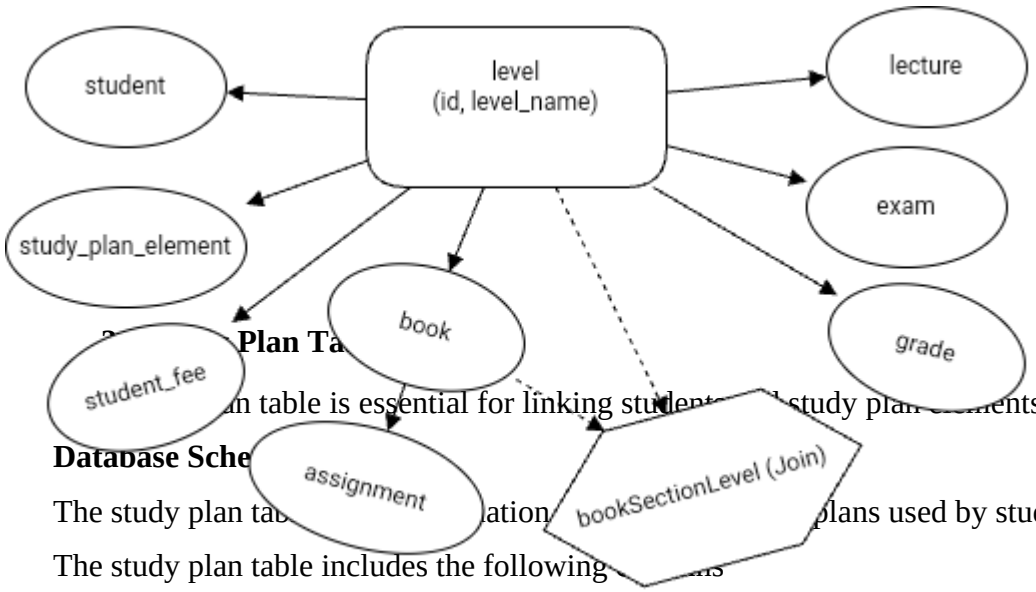
level.hasMany(models.grade, { foreignKey: 'level_id' }).

7.Many-to-Many Relationship (Level - Book)

A level can have multiple books associated with it through a joining table bookSectionLevel.

Implemented via :

level.belongsToMany(models.book, { through: 'bookSectionLevel', foreignKey: 'levelId' }).



Plan Ta

n table is essential for linking stu study pla ents.

Database Sche

The study plan ta ation plans used by students.

The study plan table includes the following

Column Name	Data Type	Description

study_plan_id	INTEGER	(PK)Unique identifier for each study plan.
study_plan_name	STRING	Name or identifier for the academic study plan.

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established:

1. One-to-Many Relationship (Study Plan - Student)

Each study plan can be associated with many students.

Implemented via :

study_plan.hasMany(models.student, { foreignKey: 'study_plan_id' }).

2.One-to-Many Relationship (Study Plan - Study Plan Element)

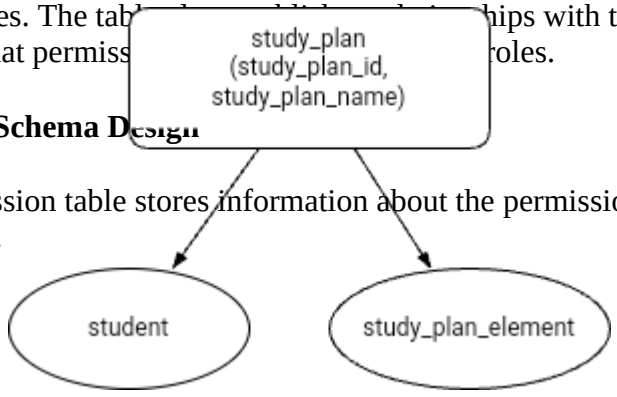
Each study plan can have multiple study plan elements associated with it.

Implemented via :

study_plan.hasMany(models.study_plan_element, { foreignKey: 'study_plan_id' }).

4. Permission Table

The permission table contains details about specific actions that can be performed on defined targets, enabling the control of system functionality based on user roles. The tab hips with the role table, ensuring that permiss roles.



Database Schema D

The permission table stores information about the permissions granted within the system.

The Permission table includes the following columns

Column Name	Data Type	Description
id	INTEGER	(PK)Unique identifier for each permission.
target	STRING	The target of the permission (e.g., "user", "product").
action	STRING	The action that can be performed on the target (e.g., "create", "update").

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established:

1.Many-to-Many Relationship (Permission - Role)

A permission can be associated with multiple roles, and a role can have multiple permissions.

Implemented via :

permission.belongsToMany(models.role, { through: 'role_permission', foreignKey: 'permissionId' }).

Data Integrity and Constraints

The permission table ensures uniqueness on the combination of target and action, applying a unique constraint to avoid duplicate permissions for the same target and action

5. Role Table

The role table is central to managing access control in the system, as it defines the different roles available, such as Admin, User, or Manager. These roles are then associated with permissions to determine what actions can be performed within the system.

Database Schema Design

The role table is designed to store details about the various roles that can be assigned to users. Each role will have a name and will be linked to multiple permissions, defining what actions are allowed for users with that role.

The role table includes the following columns:

Column Name	Data Type	Description
id	INTEGER	(PK)Unique identifier for each role.
roleName	STRING	Name of the role (e.g., "Admin", "User").

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established:

1.Many-to-Many Relationship (Role - Permission)

A role can have multiple permissions, and a permission can be assigned to multiple roles.

Implemented via :

role.belongsToMany(models.permission, { through: 'role_permission', foreignKey: 'roleId' }).

2.One-to-Many Relationship (Role - User)

A role can be assigned to multiple users.

Implemented via :

role.hasMany(models.user, { foreignKey: 'roleId' }).

Data Integrity

The roleName field is unique, ensuring that role names do not repeat. This guarantees that each role has a distinct identifier.

Access Control and Security

The role table plays a crucial role in controlling access within the system. By assigning roles to users and associating them with specific permissions, the system ensures that only authorized users can perform certain actions.

6. Role permission Table

This section focuses on the design of the role permission table, which acts as a junction table for the many-to-many relationship between roles and

```
roleId: {
  type: DataTypes.INTEGER,
  allowNull: false,
  references: {
    model: 'roles',
    key: 'id',
  },
  onDelete: 'CASCADE',
  onUpdate: 'CASCADE',
},
```

es assigning multiple permissions to different agement of user access.

signed to establish a many-to-many relationship n tables. Each record in this table links a specific role to a specific permission.

The role permission table includes the following columns

Column Name	Data Type	Description
id	INTEGER	(PK)Unique identifier for each entry.
roleId	INTEGER	Foreign key referencing the role table.
permissionId	INTEGER	Foreign key referencing the permission table.

Foreign Keys and Constraints

Foreign Key Constraints:

The roleId references the role table, ensuring that each permission is linked to a valid role.

The permissionId references the permission table, ensuring each permission is associated with a valid permission entry.

Both foreign keys use the CASCADE option for both onDelete and onUpdate to maintain referential integrity when roles or permissions are updated or deleted.

Database Relationships

This table serves as a junction between the role and permission tables, establishing a many-to-many relationship. A role can have multiple permissions, and a permission can be assigned to multiple roles.

Data Integrity

The role permission table ensures that the data remains consistent and follows relational database principles. By using foreign keys and cascading actions, the integrity of role-permission associations is maintained

7. User Table

The user table is a central table that stores key information about system users.

Database Schema Design

The user table includes the following columns:

Column Name	Data Type	Description
user_id	INTEGER	(PK) Unique identifier for each user.
user_name	JSON	Stores the user's name in multiple languages.
user_section_id	INTEGER	(FK) References the section table.
date_of_birth	DATE	Stores the user's birth date.
profile_picture	STRING	Stores the path to the user's profile image.
collegeName	JSON	Stores the name of the college in different languages.
email	STRING (Unique)	The user's email address.
roleId	INTEGER	(FK) References the roles table.
password	STRING	Encrypted user password using `bcrypt`.
resetToken	TEXT (Nullable)	Token for password reset functionality.
refreshToken	TEXT (Nullable)	Token used for session management.

Database Relationships

To ensure data consistency and efficient linking between entities, multiple relationships were established:

1.One-to-One Relationship (User - Student)

Each user can have a corresponding student profile.

Implemented via :

`hasOne(models.student, { foreignKey: 'student_id' })`.

2.One-to-Many Relationship (User - Phone Numbers)

A user can have multiple registered phone numbers.

Implemented via :

`hasMany(models.phone_number, { foreignKey: 'user_id' })`.

3.One-to-One Relationship (User - Doctor)

Each user can have a corresponding doctor profile.

Implemented via :

`hasOne(models.doctor, { foreignKey: 'doctor_id' })`.

4.One-to-Many Relationship (User - Notifications)

A user can send multiple notifications.

Implemented via :

`hasMany(models.notification, { foreignKey: 'sender_id' })`.

5. One-to-Many Relationship (User - Books)

A user can add multiple books.

Implemented via :

`hasMany(models.book, { foreignKey: 'added_by' })`.

6. Many-to-One Relationship (User - Section)

Each user belongs to a specific section.

Implemented via :

`belongsTo(models.section, { foreignKey: 'user_section_id' })`.

7. Many-to-One Relationship (User - Role)

Each user is assigned a specific role in the system.

Implemented via :

`belongsTo(models.role, { foreignKey: 'roleId' })`.

Database Security Measures

To protect sensitive data, various security measures were implemented:

- ★ Default Scope and Scoes By default field like password is excluded.
- ★ Password Hashing using `bcrypt` to encrypt user passwords.
- ★ Access Control through `roleId` to define different user permissions.
- ★ Token-based Authentication using `JWT` for secure session management.

8. Subject Table

The subject table represents academic subjects or courses offered by an institution. It has several relationships with other tables, making it a central part of the database structure.

Database Schema Design

This table stores information about subjects offered in the academic program.

The subject table includes the following columns

Column Name	Data Type	Description
subject_id	STRING(10)	(PK)A unique identifier for each subject, such as "MATH101".
subject_name	JSON	The name of the subject, potentially stored in multiple languages.
number_of_units	TINYINT	The number of academic units or credits assigned to the subject.
subject_description	JSON	A description of the subject, stored in multiple languages if needed.

Database Relationships

- One-to-Many Relationships:

study_plan_elment table: Each subject can be part of many study plan elements.

grade table: Each subject can have multiple grade entries.

prerequisite table: A subject can have multiple prerequisites.

exam table: A subject can have multiple associated exams.

book table: A subject can have multiple books associated with it.

assignment table: A subject can have multiple assignments.

- Many-to-Many Relationship:

doctor table: Each subject can have multiple doctors (teachers) associated with it through a junction table subject_teacher.

9. Student Table

The student table holds data about students, including their user details, academic information, enrollment year, and the courses they're registered for.

Database Schema Design

The student table stores personal and academic data about the students.

The student table includes the following columns:

Column Name	Data Type	Description
student_id	INTEGER	(PK) and (FK) The unique identifier for each student, linked to the users table.
study_plan_id	INTEGER	The ID of the student's study plan, linked to the study_plans table.
student_level_id	INTEGER	The student's level ID, linked to the levels table.
enrollment_year	DATEONLY	Student enrollment year.
student_system	JSON	System data related to the student, such as (General,FreeSeat,Paid), potentially stored in multiple languages.

repeat_years_count	INTEGER	The number of years the student has repeated.

Database Relationships

- One-to-One Relationships:

user table: Each student is linked to a user, as the student_id in the student table is a foreign key referring to the user_id in the user table.

- One-to-Many Relationships:

study_plan table: Each student can have one study plan. The study_plan_id is a foreign key in the student table referring to the study_plans table.

grade table: Each student can have multiple grades, associated by the student_id foreign key in the grade table.

student_fee table: A student can have multiple fee records, with the student_id in the student_fee table.

student_assignment table: Each student can have multiple assignments. The student_id is used to reference the student in the student_assignment table.

- Many-to-Many Relationships:

assignment table: Students are linked to assignments via a many-to-many relationship through the student_assignment junction table.

Default Scope and Exclusion

The default scope excludes certain attributes (like study_plan_id and student_level_id) from being returned in the response by default. However, it still includes related models such as study_plan and level.

```
101     defaultScope: {
102       attributes: { exclude: ['study_plan_id', 'student_level_id'] },
103       include: [
104         {
105           association: 'study_plan',
106
107         },
108         {
109           association: 'level',
110
111         }
112       ],
113     },
114
```

Security and Integrity

Cascading Deletes and Updates:

The foreign keys in the student table are set to CASCADE for updates and deletes, ensuring that changes made to associated records (like users or study plans) are reflected in the student data.

```
student_id: {
  type: DataTypes.INTEGER,
  primaryKey: true,
  references: {
    model: 'users',
    key: 'user_id',
  },
  onDelete: 'CASCADE',//if a user is delete the student associated with him will be deleted
  onUpdate: 'CASCADE',//if a user is update the student associated with him will be updated
},
```

Null Handling for Study Plan:

If a study plan is deleted, the study_plan_id for the student is set to NULL, ensuring data integrity without losing student records.

```
study_plan_id: {
  type: DataTypes.INTEGER,
  allowNull: true,
  references: {
    model: 'study_plans',
    key: 'study_plan_id',
  },
  onDelete: 'SET NULL',//if a study_plan is delete the student associated with him will be set null in
  onUpdate: 'CASCADE',//if a study_plan is update the student associated with him will be updated
},
```

Methods and Extra Functionality

getFullData Method:

This method merges the student's data with the related user data. If a student has an associated user, it combines the student and user data and returns it as a single object, excluding unnecessary fields like user.

```
118
119  student.prototype.getFullData = function () {
120    const student = this.toJSON();
121    if (this.user) {
122      delete student.user;
123      return { ...this.user.toJSON(), ...student };
124    }
125    return student;
126  };
```

10. Phone number Table

The phone number table stores phone numbers for users, allowing multiple phone numbers to be associated with a single user.

Database Schema Design

The phone number table stores phone numbers, with each number linked to a specific user.

The phone number table includes the following columns

Column Name	Data Type	Description
user_id	INTEGER	(FK)The ID of the user to whom the phone number belongs,Linked to users.
phone_number	STRING(25)	The phone number associated with the user.

Database Relationship

One-to-Many Relationship:

user table: The phone_number table has a foreign key user_id that references the user_id in the user table. This allows a user to have multiple phone numbers.

Cascading Updates and Deletes

The user_id foreign key in the phone_number table is set to CASCADE for both onDelete and onUpdate. This ensures that if a user is deleted or updated, the related phone numbers are also deleted or updated accordingly.

11. doctor Table

The doctor table stores information about the doctors, each of whom is associated with a user record and can have multiple subjects, assignments, and study plan elements.

Database Schema Design

The doctor table stores personal and professional information about doctors and teaching staff, with a reference to the user table for user-related details.

The doctor table includes the following columns

Column Name	Data Type	Description
doctor_id	INTEGER	(PK) and (FK) The ID of the doctor, referencing the user_id from the users table.
academic_degree	JSON	The academic degree(s) of the doctor. potentially stored in multiple languages.
administrative_position	JSON	The administrative position of the doctor (default is 'None'). potentially stored in multiple languages.

Database Relationship

- One-to-One Relationship:

user table: The doctor table has a foreign key doctor_id that references user_id in the user table, meaning that each doctor corresponds to a single user record.

- One-to-Many Relationships:

study_plan_elment: A doctor can have many study plan elements assigned to them, linked via doctor_id.

assignment: A doctor can be assigned many assignments, with each assignment associated with a specific doctor via doctor_id.

- Many-to-Many Relationship:

subject table: A doctor can teach many subjects and vice versa. This relationship is managed through a junction table called subject_teacher, with doctor_id linking doctors and subjects

12. Subject teacher Table

The subject teacher table stores the relationship between doctors and the subjects they teach. This table connects the subjects table with the doctors table.

Database Schema Design

The subject teacher table is a junction table used to establish a many-to-many relationship between subjects and doctors.

The subject teacher table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the relationship.
subject_id	STRING(10)	The ID of the subject being taught, referencing subject_id in the subjects table.
doctor_id	INTEGER	The ID of the doctor teaching the subject, referencing doctor_id in the doctors table.

Foreign Keys and Relationships

Foreign Keys :

- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.

Many-to-Many Relationship

This table serves as a junction between the subject and doctors, establishing a many-to-many relationship. A subject can have multiple doctors, and a permission can be assigned to multiple subjects

13. Grade Table

The grade table stores the grades of students across different subjects, with information about terms, sections, and levels.

Database Schema Design

The grade table includes the following columns

Column Name	Data Type	Description
student_id	INTEGER	A unique identifier for the grade record (Primary Key).

student_id	INTEGER	The ID of the student who receives the grade, referencing student_id in the students table.
subject_id	STRING(10)	The ID of the subject being graded, referencing subject_id in the subjects table.
exam_grade	TINYINT	The grade received by the student in the exam (optional).
work_grade	TINYINT	The grade received by the student for coursework or assignments (optional).
term	ENUM	The term in which the grade was issued (either 'Term 1' or 'Term 2').
section_id	INTEGER	The ID of the section for which the grade is issued, referencing id in the sections table.
level_id	INTEGER	The academic level of the student, referencing id in the levels table.
year_of_issue	DATEONLY	The year in which the grade was issued.
is_absent	BOOLEAN	Indicates whether the student was absent (true if absent, false if not).
status	JSON	Indicates whether the student is a Freshman or a Repeater. potentially stored in multiple languages.

Foreign Key and Relationships

Foreign Keys:

- student_id references the student_id in the students table.
- subject_id references the subject_id in the subjects table.
- section_id references the id in the sections table.
- level_id references the id in the levels table.

Cascading Updates and Deletes

If a student , subject , section, or level is deleted or updated, the associated records in the grade table will also be deleted or updated due to the CASCADE rules on the foreign keys.

14. Study plan element Table

The study plan element table stores data about each element of the study plan, including subjects, doctors, sections, levels, and terms.

Database Schema Design

The study plan element table includes the following columns:

Column Name	Data Type	Description
study_plan_elment_id	INTEGER	A unique identifier for the study plan element (Primary Key).
study_plan_id	INTEGER	The ID of the study plan this element belongs to, referencing study_plan_id in the study_plans table.
subject_id	STRING(10)	The ID of the subject, referencing subject_id in the subjects table.
doctor_id	INTEGER	The ID of the doctor assigned to the subject, referencing doctor_id in the doctors table.
section_id	INTEGER	The ID of the section, referencing id in the sections table.
level_id	INTEGER	The academic level associated with the study plan element, referencing id in the levels table.
number_of_units	INTEGER	The number of units or credits associated with this study plan element.
term	ENUM	The term in which the study plan element is to be taught (either 'Term 1' or 'Term 2').

Foreign Key and Relationships

Foreign Keys:

- study_plan_id references the study_plan_id in the study_plans table.
- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.
- section_id references the id in the sections table.
- level_id references the id in the levels table.

Cascading Updates and Deletes:

If a study_plan, subject, doctor, section, or level is deleted or updated, the associated records in the study_plan_element table will be deleted or updated according to the CASCADE rules

15. Prerequisite Table

The prerequisite table represents the prerequisites for subjects in the study plan. It links subjects with study plan elements, indicating the necessary subject dependencies for each element.

Database Schema Design

The prerequisite table links the subject with study_plan_element to define subject prerequisites.

The prerequisite table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the prerequisite record (Primary Key).
subject_id	STRING(10)	The ID of the prerequisite subject, referencing subject_id in the subjects table.
study_plan_elment_id	INTEGER	The ID of the study plan element this prerequisite belongs to, referencing study_plan_elment_id in the study_plan_elements table.

Foreign Key and Relationships

Foreign Keys :

- subject_id references the subject_id in the subjects table.
- study_plan_elment_id references the study_plan_elment_id in the study_plan_elments table.

Cascading Updates and Deletes :

If a subject or study_plan_element is deleted or updated, the corresponding prerequisite records will be deleted or updated accordingly, based on the CASCADE rule.

Unique Constraints

The combination of study_plan_elment_id and subject_id must be unique to avoid duplicate records for prerequisites.

16. Student fee Table

The student fee table tracks the payment status of students' fees for each term and level. It associates with the student table and level table, managing details about the amount paid and the remaining amount.

Database Schema Design

The student fee table manages students' fee details, including total fees, paid amounts, and remaining fees for a specific term and level.

The student fee table includes the following columns:

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the fee record (Primary Key).
student_id	INTEGER	student's ID, referencing student_id in the students table.
level_fees_id	INTEGER	The level of the student, referencing id in the levels table.

term	ENUM('Term 1', 'Term 2')	The term for which the fee is applicable (either Term 1 or Term 2).
total_amount	DECIMAL(8, 2)	The total fee amount for the student in the given term and level.
amount_paid	DECIMAL(8, 2)	The amount already paid by the student for the term and level.
remaining_amount	DECIMAL(8, 2)	The remaining fee amount the student still owes.
payment_date	DATE	The date on which the fee was paid (nullable).
receipt_number	STRING	The receipt number associated with the payment.

Foreign Key and Relationships

- student_id references the student_id in the students table.
- level_fees_id references the id in the levels table.

Cascading Updates and Deletes

If a student or level record is deleted or updated, the related student_fee records will be deleted or updated accordingly based on the CASCADE rule

17. Lecture Table

The lecture table stores information about lectures, including the subject, doctor, section, level, and schedule details, along with the relationships for replacing lectures and tracking their status.

Database Schema Design

The lecture table tracks lectures assigned to specific subjects, doctors, sections, and levels, including schedule details like time, day, and room.

The lecture fee table includes the following columns:

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the lecture (Primary Key).

subject_id	STRING(10)	The subject associated with the lecture, referencing subject_id in the subjects table.
doctor_id	INTEGER	The doctor assigned to the lecture, referencing doctor_id in the doctors table.
lecture_section_id	INTEGER	The section in which the lecture takes place, referencing id in the sections table.
lecture_level_id	INTEGER	The level of the lecture, referencing id in the levels table.
term	ENUM('Term 1', 'Term 2')	The term for which the lecture is scheduled (either Term 1 or Term 2).
year	STRING(30)	The academic year during which the lecture is given.
lecture_time	TIME	The time at which the lecture takes place.
lecture_duration	INTEGER	The duration of the lecture in minutes.
lecture_day	ENUM	The day of the week on which the lecture occurs.
lecture_room	STRING(50)	The room where the lecture takes place (nullable).
lectureStatus	BOOLEAN	Specifies the lecture status(default is `true`).
isReplaced	BOOLEAN	Indicates whether the lecture has been replaced (default is `false`).
originalLecturId	INTEGER	The ID of the original lecture if this lecture replaces another, referencing id in the lectures table (nullable).

Foreign Key and Relationships

- subject_id references the subject_id in the subjects table.
- doctor_id references the doctor_id in the doctors table.
- lecture_section_id references the id in the sections table.
- lecture_level_id references the id in the levels table.

Self-referencing Relationships

- originalLecturId references the id in the lectures table.

Implemented via :

```
lecture.belongsTo(models.lecture, {
  foreignKey: 'originalLecturId',
  as: 'originalLecture',
});
```

Lecture Replacement System

- If a lecture is rescheduled or replaced, it refers back to original lecture :

  Lecture.originalLctureId→Lecture.id

- The alias **originalLecture** is used for the **originalLecture**.
- The alias **replacedLecture** hold all lectures replacing aspecific one.

Cascading Updates and Deletes:

- If a subject, doctor, section, or level record is deleted or updated, the related lecture record will be updated or deleted according to the CASCADE rule.
- Special Relationship for Replacing Lectures:
- originalLecturId links to another lecture if it replaces an existing one. The replaced lecture is tracked in the `replacedLecture` association.

Indexing

To improve performance and enforce uniqueness on certain fields, **unique index** is defined on the lecture table.

This index ensures that on two lectures can have the same **time, day, section, room,** and **replacement status** simultaneously.

This is crucial to prevent scheduling conflicts in the same section and room

Defined index in model :

```
  indexes: [
      {
        unique: true,
        fields: ['lecture_time', 'lecture_day',
'lecture_section_id', 'lecture_room','isReplaced'],
        name: 'unique_constraint_in_lecture',
      },
    ],
```

Defined index in migration :

```
    await queryInterface.addConstraint('lectures', {
      fields: ['lecture_time', 'lecture_day',
'lecture_section_id', 'lecture_room','isReplaced'],
      type: 'unique',
      name: 'unique_constraint_in_lecture',
    });
```

18. Exam Table

The exam table stores the details of exams including the subject, section, level, and specific details like exam date, time, and room. It defines the relationships between different entities, ensuring that exams are linked to specific subjects, sections, and levels.

Database Schema Design

The exam table is used to store details about each exam, including which subject, section, and level it is associated with, along with the exam's scheduling details.

The exam table includes the following columns

Column Name	Data Type	Description
exam_id	INTEGER	A unique identifier for the exam (Primary Key).
subject_id	STRING(10)	The subject associated with the exam, referencing subject_id in the subjects table.
exam_section_id	INTEGER	The section for the exam, referencing id in the sections table.
exam_level_id	INTEGER	The level for the exam, referencing id in the levels table.
exam_date	DATEONLY	The date when the exam is scheduled.
exam_time	TIME	The time when the exam starts.
exam_day	ENUM	The day of the week on which the exam occurs.
exam_room	STRING(50)	The room where the exam will take place.

Foreign Key and Relationships

- subject_id references subject_id in the subjects table.
- exam_section_id references id in the sections table.
- exam_level_id references id in the levels table.

Cascading Updates and Deletes

If a subject, section, or level record is deleted or updated, the related exam record will be updated or deleted according to the CASCADE rule.

Unique Constraints

The combination of exam_section_id, exam_level_id, exam_term, exam_year, exam_date, exam_time, and exam_day must be unique to prevent scheduling conflicts.

19. Notification Table

The notification table stores notifications sent to users. It defines the relationship between notifications and users, capturing details such as the message content, sender, and notification status.

Database Schema Design

The notification table stores information about notifications, including the sender, message, title, and notification type.

The notification table includes the following columns

Column Name	Data Type	Description
message_id	INTEGER	A unique identifier for the notification (Primary Key).
sender_id	INTEGER	The user who sent the notification, referencing user_id in the users table.
title	STRING(100)	The title of the notification.
message	TEXT	The content of the notification.
is_read	BOOLEAN	A flag indicating whether the notification has been read (default: `false`).
type	ENUM('System', 'Reminder', 'Alert')	The type of notification (default: `System`).

Foreign Key and Relationships

- sender_id references user_id in the users table.

Cascading Updates and Deletes

If a user record is deleted or updated, the related notification records will be updated or deleted according to the CASCADE rule.

20. Book Table

The book table is designed to manage and store information about books in a system. It defines relationships between books, users, subjects, levels, and sections. The table includes various fields to store book details such as the title, author, file information, and the user who added the book.

Database Schema Design

The book table includes the following columns:

Column Name	Data Type	Description

id	INTEGER	A unique identifier for the book (Primary Key).
section_id	INTEGER	The ID of the section the book belongs to (Foreign Key from sections).
level_id	INTEGER	The ID of the level the book is associated with (Foreign Key from levels).
title	STRING	The title of the book.
author	STRING	The author of the book (optional).
numberOfPages	INTEGER	The number of pages in the book (optional).
edition	STRING(50)	The edition of the book (optional).
category	ENUM	The category of the book (e.g., Book, Reference, etc.).
file_size	FLOAT	The file size of the book in bytes (optional).
file_path	TEXT	The path to the book file.
display_image	STRING	The image representing the book (optional).
added_by	INTEGER	The ID of the user who added the book (Foreign Key from users).
subject_id	STRING(10)	The ID of the subject the book is related to (Foreign Key from subjects).
original_name	STRING	The original name of the book file.

Foreign Key and Relationships

- `section_id` references `id` in the sections table.
- `level_id` references `id` in the levels table.
- `added_by` references `user_id` in the users table.
- `subject_id` references `subject_id` in the subjects table.

Many-to-Many Relationship:

The book table has many-to-many relationships with the level and section tables through the bookSectionLevel join table.

Cascading Updates and Deletes

If a referenced record in the users, subjects, levels, or sections table is deleted or updated, related records in the book table are updated or deleted accordingly.

21. Book Section Level Table

The bookSectionLevel table is a join table designed to represent the many-to-many relationships between books, sections, and levels. It helps link books to specific sections and levels, and ensures that each book can be associated with multiple sections and levels.

Database Schema Design

The Book Section Level table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
bookId	INTEGER	The ID of the book (Foreign Key from books).
sectionId	INTEGER	The ID of the section (Foreign Key from sections).
levelId	INTEGER	The ID of the level (Foreign Key from levels).

Foreign Key and Relationships

- `bookId` references the `id` in the books table.
- `sectionId` references the `id` in the sections table.
- `levelId` references the `id` in the levels table.

Unique Constraints

A unique constraint is applied on the combination of `bookId`, `sectionId`, and `levelId` to prevent duplicate associations between the same book, section, and level.

22. **Assignment**

The assignment table is used to store information about assignments. It includes relationships with multiple tables such as subjects, doctors, students, sections, and levels. Each assignment is linked to a specific subject, doctor, section, and level, with additional information like title, due day, and dates.

Database Schema Design

The assignment table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
subject_id	STRING(10)	The ID of the subject (Foreign Key from subjects).
doctor_id	INTEGER	The ID of the doctor (Foreign Key from doctors).
section_id	INTEGER	The ID of the section (Foreign Key from sections).
level_id	INTEGER	The ID of the level (Foreign Key from levels).
title	JSON	The title of the assignment (can be multi-lingual)
assignment_due_day	ENUM	The day of the week the assignment is due (e.g., Saturday).
assignment_date	DATE	The date the assignment was created
assignments_due_date	DATE	The deadline for the assignment.

Foreign Key and Relationships

- `subject_id` references the `subject_id` in the subjects table.
- `doctor_id` references the `doctor_id` in the doctors table.
- `section_id` references the `id` in the sections table.
- `level_id` references the `id` in the levels table.

Many-to-Many Relationship

students are related to assignments through the student_assignment table (many-to-many relationship).

23. Student Assignment Table

The student assignment table represents the relationship between students and assignments, specifically tracking which student is assigned to which assignment and the status of their submission. This table also tracks additional information, such as whether the assignment has been completed and any related files.

Database Schema Design

The Student Assignment table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
student_id	INTEGER	The ID of the student (Foreign Key from students).
assignment_id	INTEGER	The ID of the assignment (Foreign Key from assignments).
status	ENUM	The status of the assignment submission (Accepted, Rejected, etc.).
is_completed	BOOLEAN	Whether the student has completed the assignment.

Foreign Key and Relationships

- `student_id` references the `student_id` in the students table.
- `assignment_id` references the `id` in the assignments table.

Relationships:

- A student can have multiple assignments, and an assignment can be assigned to many students, so we use a many-to-one relationship between student_assignment and both the students and assignments tables.

```
student_assignment.belongsTo(models.assignment, {
  foreignKey: 'assignment_id',
});

student_assignment.belongsTo(models.student, {
  foreignKey: 'student_id',
});
```

- The student_assignment table also has a one-to-many relationship with the student_assignment_file table, allowing multiple files to be associated with each student's assignment.

```
student_assignment.hasMany(models.student_assignment_file,
{
    foreignKey: 'student_assignment_id',
 });
```

24. Assignment File Table

The assignment file table is designed to store files related to assignments. This table includes metadata about each file, such as its attachment, a hash for integrity validation, and the original file name. It is connected to the assignment table through a foreign key.

Database Schema Design

The assignment file table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).
assignment_id	INTEGER	The ID of the assignment (Foreign Key from assignments).
attachment	TEXT	The file's attachment (usually in base64 format or file path).
attachment_hash	STRING(64)	A hash value of the file for validation purposes.
original_name	STRING	The original name of the file as uploaded by the user.

Foreign Key and Relationships

- `assignment_id` references the `id` in the assignments table.

Relationship

The assignment_file table is linked to the assignments table with a one-to-many relationship, meaning each assignment can have multiple files attached.

25. Student Assignment File Table

The student assignment file table is designed to store files related to student assignments. This table includes metadata about each file, such as its attachment, a hash for file integrity, and the original file name. It is associated with the student assignment table through a foreign key.

Database Schema Design

The student assignment file table includes the following columns

Column Name	Data Type	Description
id	INTEGER	A unique identifier for the record (Primary Key).

student_assignment_id	INTEGER	The ID of the student assignment (Foreign Key from student_assignments).
attachment	TEXT	The file's attachment (usually in base64 format or file path).
attachment_hash	STRING(64)	A hash value of the file for integrity validation.
original_name	STRING	The original name of the file as uploaded by the student.

Foreign Key and Relationships

- `student_assignment_id` references the `id` in the tudent_assignments table.

Relationship

The student_assignment_file table is linked to the student_assignments table with a one-to-many relationship, meaning each student assignment can have multiple files attached.

26. Refresh State Table

The refresh state table is used to store the state of a refresh operation, potentially for an application feature like session management or data filtering. It keeps track of the target of the refresh, the filter used (if any), and uses a unique identifier as the primary key.

Database Schema Design

The refresh state table includes the following columns

Column Name	Data Type	Description
id	STRING	A unique identifier for the record (Primary Key).
target	STRING	The target entity for which the refresh state is being tracked.

filter	JSON	A JSON field storing any filtering criteria for the refresh.

Foreign Key and Relationships

Foreign Key

- The refresh_state table does not include any foreign key relationships as it operates independently.

Relationships

There are no specific relationships defined for this table.